



Gustave Eiffel University

Internship Report
Master 2 Bézout

Fine-Grained Analysis of the Hyperscan Engine

CHAU Dang Minh

Supervisors: Prof. Cyril NICAUD (LIGM)
Dr. Pablo ROTONDO (LIGM)

Paris, August 2026



Acknowledgements

I would like to begin by expressing my deep gratitude to my supervisors, Prof. Cyril Nicaud and Dr. Pablo Rotondo. Their support, insightful guidance, and generous patience have been instrumental throughout this work. With their depth of experience and dedication, they have consistently offered thoughtful feedback, challenged my assumptions, and encouraged me to grow as an independent researcher. Their mentorship has extended well beyond academic supervision. They observed my progress closely and always encouraged me to keep going. I am deeply thankful for their mentorship and for the invaluable lessons they have imparted.

My heartfelt thanks go as well to Prof. Laurent Hauswirth, Prof. Marie-Pierre Béal, Prof. Matthieu Fradelizi, Prof. Jean-Christophe Novelli, Dr. Olivier Sester, Dr. Miguel Martinez, Dr. Johan Thapper, Dr. Nicolas Borie, Dr. Pierre-André Zitt, Dr. Thomas Bonis, Dr. Théo Lacombe, Dr. Olivier Guédon, and Dr. Alfredo Hubard. Their expertise and teaching over the past year have enriched my understanding and strengthened my foundation in mathematics, preparing me for the internship and for the beginning of my research career. It was during this internship that I realized how deeply connected seemingly disparate areas of mathematics can be when solving practical problems. Knowledge from symbolic dynamics, convex geometry, theoretical machine learning, and optimization has been useful in understanding the Hyperscan engine, both through specific techniques and through methods of reasoning.

Finally, I wish to acknowledge the unwavering love and support of my family and close friends, near and far. To my mother, father, sister, and to Huynh, my girlfriend—your encouragement has been my anchor. To Nam, Nguyen, Luan, and Quyen—thank you for being my constant companions. Your presence has made this journey not only possible but meaningful.

Paris, August 27th, 2025

Chau Dang Minh

Contents

List of Figures	5
1 Introduction	6
2 Concepts and Techniques	7
2.1 Regular expressions	7
2.2 SIMD parallelism	9
2.3 Analytic combinatorics	10
2.4 Monte Carlo simulation	10
3 The Hyperscan Engine and Related Problems	12
3.1 Overview of Hyperscan	12
3.1.1 Compilation	12
3.1.2 Execution	14
3.2 The FDR string matcher	15
3.2.1 The Shift-Or algorithm	15
3.2.2 Compilation	16
3.2.3 Execution	17
3.3 Related problems	20
3.3.1 Optimizing regular expression decomposition	20
3.3.2 Analysis of the FDR string matcher	20
3.3.3 Statistics of regular expressions	20
4 The Length of Longest Literals	22
4.1 Lower and upper bounds	22

4.2	Asymptotics of the lower bound recurrence	23
4.3	Asymptotics of the stochastic recurrence	34
4.3.1	Proof strategy	34
4.3.2	A fixed-point approach: existence	37
5	Conclusion	39
A	Appendix	41
A.1	Supplementary results	41
A.1.1	Asymptotics of recurrences that include a sum of history	41
A.1.2	Spaces of probability measures	42
A.2	Remaining proofs	43
A.2.1	Lemma 4.13	43
A.2.2	Proposition 4.14	45
A.2.3	Proposition 4.16	45
A.2.4	Proposition 4.18	46
A.2.5	Proposition 4.19	46
B	Bibliography	49

List of Figures

2.1	DFA recognizing strings over $\{a, b\}$ that do not contain aa	8
3.1	Dependency graph of main functions in the compilation phase.	13
3.2	The dependency graphs by Violet and Rose.	14
3.3	Dependency graph of main functions in the execution phase.	14
3.4	Proposed sequential dependency graph representation for <code>abc[0-9]+xyz(de)*</code>	20
4.1	Computation of z_n/n^α	24
4.2	First consecutive differences of (z_n) in two initializations ($u = 0.5000001, v = 0.000001$)	28
4.3	The growth of y_n compared with a power Cn^α	35
4.4	Results of the mean-preserving recurrence (4.20) with $u = 0.5$ and $v = 0.3$	36

1

Introduction

Since Knuth popularized the term *analysis of algorithms* [9, Preface], the field has grown to include highly specialized subfields. One useful way to classify such analyses is by two aspects. The first is the scenario under which the performance of an algorithm is analyzed, such as worst-case or average-case analysis. In many cases, average-case analysis is more informative than worst-case analysis, and is often more challenging to perform, since it requires a probabilistic model of the input that is both tractable and expressive enough. For example, in the case of regular languages, generating syntax trees uniformly at random may be unsuitable [10]. Average-case analysis also opens the door to techniques from other fields, such as probability theory and analytic combinatorics, and raises new questions, including questions about limiting objects as the input size tends to infinity. The second aspect is the underlying cost model, such as the uniform model or the word-RAM model. A cost model should align quantitative analysis with practical performance, for example execution time. The analysis of modern algorithms may therefore require cost models that capture hardware features such as caching, branch prediction, and parallelism.

In this internship, we study Hyperscan, a regular expression matching engine that uses SIMD parallelism to achieve high performance. Because Hyperscan has a sophisticated implementation, we focus on questions that can be isolated and studied mathematically. Hyperscan extracts possible strings from a regular expression and matches them in parallel using SIMD string matchers. It is therefore natural to analyze statistics related to those strings under a distribution. In particular, our main question is to understand the asymptotics of the length of the longest string extracted from a regular expression. We also formulate related questions that were not studied in depth. Details are given in the next chapter.

The rest of the report is organized as follows. **Chapter 2** introduces the concepts and techniques used in this internship, including regular expressions, BST-like distributions, SIMD parallelism, analytic combinatorics, and Monte Carlo simulation. In **Chapter 3**, we present Hyperscan and several problems arising from its analysis. **Chapter 4** contains our results on the length of the longest extracted string. Where useful, proofs are preceded by the motivation behind the approach. Finally, **Chapter 5** summarizes the work and discusses future directions.

2

Concepts and Techniques

This chapter introduces the concepts and techniques used in the report. We first review regular expressions, finite automata, the Glushkov construction, and BST-like distributions. We then discuss SIMD parallelism and its role in regular expression matching. Finally, we introduce tools from analytic combinatorics and Monte Carlo simulation, which are used to analyze asymptotic recurrences and estimate distributions. Some functional-analytic tools are introduced later in Chapter 4, when they are needed for the longest literal problem.

2.1 Regular expressions

A *regular expression* is a string that describes a set of strings according to certain syntax rules. Formally, given a finite alphabet Σ , the set of regular expressions over Σ is defined inductively. The base cases consist of the empty set $\emptyset$, the empty word ϵ , and literal characters $a \in \Sigma$. More complex expressions are constructed using three fundamental operators: alternation ($|$), concatenation (implicit or $\cdot$), and the Kleene star ($*$). Semantically, a regular expression E maps to a formal language $\mathcal{L}(E) \subseteq \Sigma^*$ via the following structural rules:

$$\begin{aligned}\mathcal{L}(\emptyset) &= \emptyset, & \mathcal{L}(\epsilon) &= \{\epsilon\}, & \mathcal{L}(a) &= \{a\}, \\ \mathcal{L}(E_1 | E_2) &= \mathcal{L}(E_1) \cup \mathcal{L}(E_2), \\ \mathcal{L}(E_1 \cdot E_2) &= \{xy \mid x \in \mathcal{L}(E_1), y \in \mathcal{L}(E_2)\}, \\ \mathcal{L}(E^*) &= \bigcup_{n \geq 0} \mathcal{L}(E)^n.\end{aligned}$$

Any language generated by these rules is called a regular language. In practice, regular expressions are often extended with additional operators, such as the plus operator ($+$), which denotes one or more repetitions, the question mark operator ($?$), which denotes zero or one occurrence, and the bounded repetition operator ($\{m, n\}$), which denotes between m and n occurrences. These extensions preserve the class of regular languages because they can be expressed using the fundamental operators: $E^+ = E \cdot E^*$, $E? = \epsilon \mid E$, and $E^{\{m, n\}} = E^m \cdot (E?)^{n-m}$.

A fundamental result in formal language theory is that the expressive power of regular expressions is identical to that of *finite automata*, which provide an operational point of view [14]. A finite automaton has a set of states, a distinguished start state, and a set of accepting states. Transitions between states are triggered by input symbols. The formal definition is as follows.

Definition 2.1. A (non-deterministic) finite automaton (NFA) is a quintuple $(\Sigma, Q, q_0, F, \delta)$ where

2.1 Regular expressions

- Σ is a finite alphabet of input symbols,
- Q is a finite set of states,
- $q_0 \in Q$ is the start state,
- $F \subseteq Q$ is the set of terminal states, and
- $\delta : Q \times \Sigma \rightarrow 2^Q$ is the transition function.

The automaton is a deterministic finite automaton (DFA) if $\delta(q, a)$ is a singleton for every $q \in Q$ and $a \in \Sigma$. ■

Example 2.2. Let $\Sigma = \{a, b\}$. The regular expression $E = (b \mid ab)^*(\epsilon \mid a)$ consists of all strings over Σ that do not contain two consecutive a 's. The corresponding DFA is shown in Figure 2.1. ■

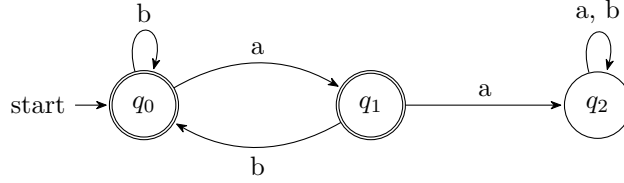


Figure 2.1: DFA recognizing strings over $\{a, b\}$ that do not contain aa .

Finite automata are basic data structures for regular expression matching. The Glushkov construction [7] used by Hyperscan is a well-known algorithm that converts a regular expression into an equivalent NFA. It has several useful properties: it has no ϵ -transitions, the number of states is one more than the number of symbols in the regular expression, and all incoming transitions to a given state read the same letter.

Average-case analysis of algorithms on regular expressions requires a probability distribution over regular expressions. More precisely, we use a distribution over syntax trees of regular expressions, which are rooted ordered trees representing the structure of a regular expression. The syntax tree of a regular expression is not unique: for example, abc has two syntax trees, each corresponding to a different bracketing of the concatenation operator.

It is known that the uniform distribution over syntax trees of regular expressions over a two-letter alphabet tends to generate reduced syntax trees of sizes bounded by 80 [11]. This suggests that the uniform distribution is not a good model for analyzing the average-case complexity of algorithms on large regular expressions. Instead, we use a generalized family of generators over binary search trees (BST), commonly called BST-like distributions. A tree of size n is generated as follows. If $n = 1$, a literal is generated uniformly at random. If $n > 1$, a node is generated with probability u or v for the concatenation or alternation operator, respectively, and with probability $1 - u - v$ for the Kleene star operator. The subtrees are then generated recursively with sizes summing to $n - 1$. The formal definition is given below, using a probability function $p : \mathcal{T} \rightarrow [0, 1]$ that maps a syntax tree to its probability of being generated, such that $\sum_{|T|=n} p(T) = 1$.

Definition 2.3 (BST-like distributions). Let u and v be parameters in $[0, 1]$ with $u + v < 1$. The function $p : \mathcal{T} \rightarrow [0, 1]$ is defined inductively as follows.

1. $\sum_{c \in \Sigma} p(c) = 1$,
2. $p\left(\begin{array}{c} \bullet \\ \swarrow \quad \searrow \\ T_1 \quad T_2 \end{array}\right) = \frac{u}{n-2} p(T_1) p(T_2)$, where $|T_1| + |T_2| = n - 1$.

$$3. \ p \left(\begin{array}{c} | \\ / \backslash \\ T_1 \quad T_2 \end{array} \right) = \frac{v}{n-2} p(T_1) p(T_2), \text{ where } |T_1| + |T_2| = n - 1.$$

$$4. \ p \left(\begin{array}{c} * \\ | \\ T \end{array} \right) = (1 - u - v) p(T).$$

■

Because the expected heights differ asymptotically under the uniform distribution and under a BST-like distribution ($\Theta(\sqrt{n})$ versus $\Theta(\log n)$), BST-like distributions do not subsume the uniform distribution. Their flexibility comes from tuning the parameters u and v to specific datasets. For example, maximum-likelihood fitting on the Snort dataset [4] yields $u \approx 0.9$ and $v \approx 0.09$.

2.2 SIMD parallelism

Single Instruction, Multiple Data (SIMD) is a hardware feature supported in modern CPUs. The model is designed to exploit fine-grained, data-level parallelism by evaluating functions over vectors rather than individual scalars. Let $\mathcal{S}$ be a space of data elements (such as a field $\mathbb{F}$ or floating-point representations $\mathbb{R}$). A traditional Single Instruction, Single Data (SISD) architecture evaluates a scalar operation $f : \mathcal{S} \rightarrow \mathcal{S}$ over a sequence of inputs by strictly executing n sequential operations, requiring time proportional to $\mathcal{O}(n)$. In contrast, a SIMD architecture formalizes execution as a vectorized mapping. Given an input vector $X = (x_1, x_2, \dots, x_w) \in \mathcal{S}^w$, where the dimension w represents the hardware's fixed vector width (or number of processing lanes), the control unit broadcasts a single instruction to apply f element-wise across the entire vector concurrently

$$F(X) = (f(x_1), f(x_2), \dots, f(x_w))$$

By executing this element-wise map, the processor performs w independent operations within a single instruction cycle. From an algebraic perspective, SIMD architecture natively implements operations on vector spaces, executing component-wise additions, scalar multiplications, and the Hadamard product $A \circ B$ in lockstep. For an algorithm processing an input array of size N , the SIMD paradigm effectively reduces the required number of instruction fetches and sequential loop iterations from N to $\mathcal{O}(\lceil N/w \rceil)$.

Modern compilers such as GCC use static analysis to perform automatic vectorization, implicitly transforming scalar loops into the $\mathcal{O}(\lceil N/w \rceil)$ vector mappings discussed above. These compiler-driven optimizations are generally limited to predictable, contiguous data access patterns. They frequently fail to vectorize algorithms with complex control flow, non-linear dependencies, or irregular state transitions.

Consequently, approaching the theoretical limits of hardware throughput often requires explicit vectorization. The regular expression engine Hyperscan, introduced in the next chapter, is a prime example. Because its underlying graph-based automata, such as the Rose and Violet engines, rely on highly irregular state-machine transitions that a compiler cannot safely auto-vectorize, Hyperscan uses manually crafted SIMD routines based on hardware intrinsics. This explicit programming model gives direct control over the parallel vector registers, allowing the engine to process multiple transition functions concurrently and achieve high-speed pattern matching.

2.3 Analytic combinatorics

The field of analytic combinatorics [6] applies tools from complex analysis to combinatorial structures and recurrences. It treats the generating functions of such objects as analytic functions, studies their singularities, and derives asymptotic estimates for the coefficients of these generating functions. Many common uses are formalized as transfer theorems. We use one of them to find the asymptotic behavior of recurrences (4.2) and (4.3). More details are given in the next chapter. We state the needed transfer theorem here.

Theorem 2.1 (Sim-transfer, [6, Corollary VI.1]) *Let $R > 1$, $0 < \phi < \pi/2$. Define the domain $\Delta := \Delta(R, \phi)$ by*

$$\Delta(R, \phi) = \{z \in \mathbb{C} : z \neq 1, |z| < R, |\arg(z - 1)| > \phi\}.$$

Let f be analytic in some domain Δ , and

$$f(z) \sim (1 - z)^{-\alpha} \text{ as } z \rightarrow 1, z \in \Delta,$$

such that $\alpha \notin \{0, -1, -2, \dots\}$. Then the coefficients of f satisfy

$$[z^n]f(z) \sim \frac{n^{\alpha-1}}{\Gamma(\alpha)}.$$

2.4 Monte Carlo simulation

The Monte Carlo method uses a population of random samples to estimate a distribution. This approach is required later to test conjectures related to recurrence (3.5) in the longest literal problem, as computing the exact PMFs and CDFs is computationally intractable. The theoretical setup is as follows. Given a distribution μ and a number of particles n , we sample

$$X_1, X_2, \dots, X_n \stackrel{\text{iid}}{\sim} \mu.$$

The empirical distribution μ_n is defined as

$$\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i}.$$

Definition 2.4 (Weak convergence of distributions). Let (M, d) be a metric space and let $\mathcal{M}$ be the Borel σ -algebra on M . Let a probability distribution be defined on $(M, \mathcal{M})$, and let $C_b(M)$ be the set of real-valued, bounded, continuous functions on M .

A sequence of distributions (μ_n) converges weakly to a distribution μ if, for all $f \in C_b(M)$, we have

$$\int f d\mu_n \rightarrow \int f d\mu.$$

■

Theorem 2.2 (Varadarajan's Theorem [15]) *The empirical distribution sequence μ_n converges weakly to μ almost surely.*

By the Continuous Mapping Theorem, continuous operations on empirical distributions preserve weak convergence.

Proposition 2.3 (Continuous Mapping) *Let $(X_i, Y_i)_{i=1}^n$ be independent and identically distributed samples from a joint distribution μ on $M \times M$, with empirical measure $\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{(X_i, Y_i)}$. Let*

2.4 Monte Carlo simulation

$f : M \times M \rightarrow M$ be a continuous function. Then,

$$\frac{1}{n} \sum_{i=1}^n \delta_{f(X_i, Y_i)} \xrightarrow{w} f_{\#} \mu$$

almost surely.

We rely on the following special cases for real-valued random variables:

1. When $f(x, y) = x + y$, we have $\frac{1}{n} \sum_{i=1}^n \delta_{X_i + Y_i} \xrightarrow{w} \mathcal{L}(X + Y)$ almost surely.
2. When $f(x, y) = \max(x, y)$, we have $\frac{1}{n} \sum_{i=1}^n \delta_{\max(X_i, Y_i)} \xrightarrow{w} \mathcal{L}(\max(X, Y))$ almost surely.
3. When $f(x, y) = xy$, we have $\frac{1}{n} \sum_{i=1}^n \delta_{X_i Y_i} \xrightarrow{w} \mathcal{L}(XY)$ almost surely.

Proposition 2.4 (Wasserstein convergence of empirical measures) *Let (μ_n) be a sequence of empirical measures such that $\mu_n \xrightarrow{w} \mu$ almost surely, and assume the p -th moment of μ is finite. Then,*

$$W_p(\mu_n, \mu) \rightarrow 0$$

almost surely.

Using the triangle inequality, we can establish the following convergence result for two sequences.

Proposition 2.5 (Convergence of Wasserstein distance) *Let $\mu_n \xrightarrow{w} \mu$ almost surely and $\nu_n \xrightarrow{w} \nu$ almost surely. Then,*

$$W_p(\mu_n, \nu_n) \rightarrow W_p(\mu, \nu)$$

almost surely.

For one-dimensional distributions, Proposition 2.5 allows us to compute the empirical Wasserstein distance in $O(n \log n)$ time by sorting the samples:

$$W_p(\mu_n, \nu_n) = \inf_{\pi \in \Pi(\mu_n, \nu_n)} \left(\int |x - y|^p d\pi(x, y) \right)^{1/p} = \left(\frac{1}{n} \sum_{i=1}^n |\hat{X}_i - \hat{Y}_i|^p \right)^{1/p}, \quad (2.1)$$

where $(\hat{X}_1, \dots, \hat{X}_n)$ and $(\hat{Y}_1, \dots, \hat{Y}_n)$ are the sorted order statistics of the samples from μ_n and ν_n , respectively.

3

The Hyperscan Engine and Related Problems

The regular expression matching engine Hyperscan [17] was primarily developed by the startup company Sensory Networks in 2008, and was later acquired and open-sourced by Intel in 2013. After Hyperscan became closed source in 2023 at version 5.4, VectorCamp forked it as Vectorscan [16] to support more architectures, including ARM. Since Vectorscan is still actively maintained in public, analyzing the core functionality inherited from Hyperscan remains valuable for understanding the design and optimization of high-performance regular expression matching engines. Hyperscan is designed to match a set of regular expressions against a stream of data, with deep packet inspection as a primary use case. It relies heavily on SIMD instructions.

This chapter is divided into three sections. The first two sections are descriptions of Hyperscan and one of its string matchers. Their mechanisms naturally lead to several problems stated in the last section.

3.1 Overview of Hyperscan

The architecture of Hyperscan includes two phases: compilation and execution. In the compilation phase, a set of regular expressions is converted into an internal database. The database is used in the execution phase to match the regular expressions against a stream of data. Hyperscan supports two modes of operation: block mode and stream mode. In block mode, the engine processes one block of data at a time. In stream mode, it maintains state across blocks, allowing continuous matching over a data stream. We focus on block mode.

3.1.1 Compilation

An overview of the compilation process is illustrated in Figure 3.1. The compilation phase provides central APIs, such as the `hs.compile` function. The function accepts a set of regular expressions. A facade object of class `NG` is initialized, which consumes a compile context of type `CompileContext`. Through the function `addExpression`, each expression is converted into a Glushkov NFA [7] (through `buildGraph`) and added to the facade (through `NG.addGraph`).

In `NG.addGraph`, the NFA is first decomposed based on its language through `calcComponents`. If its

3.1 Overview of Hyperscan

language is L , it may be split into NFAs whose languages are $L_1, L_2, \dots, L_n$ such that

$$L = L_1 \cup L_2 \cup \dots \cup L_n.$$

The components are added one by one through `addComponent`. Each component is then passed to the Violet subengine through the `doViolet` function, which calls `doInitialVioletTransform` and `RoseBuild.addRose`.

In `doInitialVioletTransform`, the graph analyses described in the Hyperscan paper—namely dominant path, dominant region, and network flow analysis—are performed. These analyses decompose the NFA into literals and sub-FAs, yielding a dependency graph that dictates the matching order. At this phase (`RoseInGraph`), each literal is a vertex and each sub-FA acts as an edge. This graph is then passed to the Rose subengine for deduplication of FAs, compilation into bytecode, and execution orchestration. Notably, in the subsequent (`RoseGraph`) representation, the sub-FAs are instead attached directly to their associated literal vertices via `left` or `suffix` properties. Multiple FAs may be merged into an FA with multiple initial states. Therefore, the `suffix` property also contains a sub-property `top` indicating which initial state should be chosen.

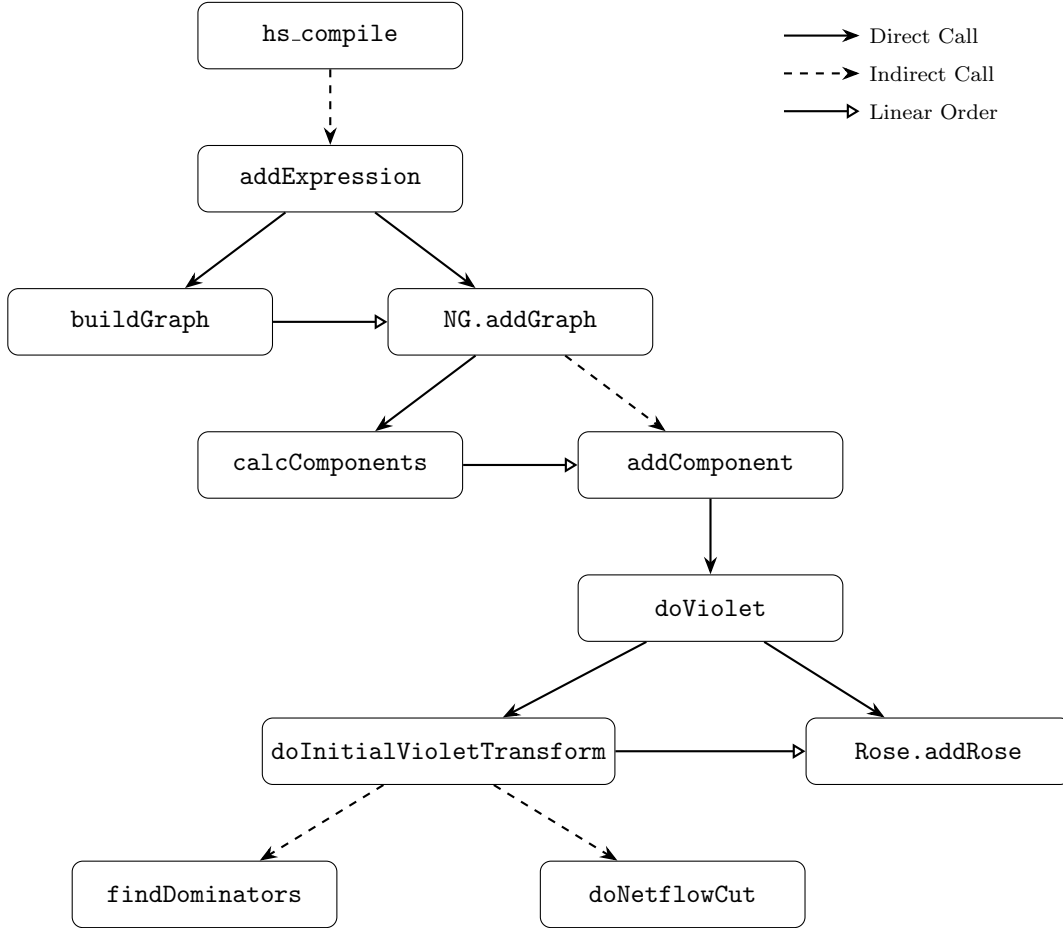


Figure 3.1: Dependency graph of main functions in the compilation phase.

Consider an example of a regular expression `abc[0-9]+xyz`. If we discard heuristics for short literals, then the graphs returned by Violet and Rose are as in Figures 3.2a and 3.2b, respectively. Both graphs encode the same matching constraint: if a match of `abc` ends at position i (excluded) and a match of `xyz` starts at position j (included), then the subtext at $[i, j)$ must match the sub-FA `[0-9]+`. For this example, the gap must contain at least one digit. More generally, Hyperscan can further impose the width constraint

$$m \leq j - i \leq M,$$

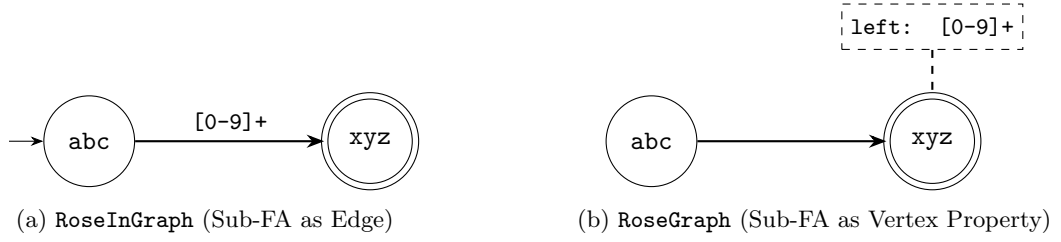


Figure 3.2: The dependency graphs by Violet and Rose.

where m and M are the minimum and maximum *widths* of the sub-FA, i.e., the minimum and maximum lengths of possible strings that match the sub-FA.

From a theoretical point of view, the edge representation and the vertex-property representation are equivalent: both describe a dependency relation between the two literals together with an intervening sub-FA that has to be checked. The difference is mainly implementation-oriented. In **RoseInGraph**, the sub-FA appears as an edge because Violet is still expressing the decomposition of the NFA. In **RoseGraph**, Rose attaches the sub-FA to the relevant literal vertex through a property such as **left** or **suffix**. This representation is more convenient for the later compilation stages, where Rose deduplicates and merges FAs, records which initial state should be used, and emits the programs that are executed when a literal match is reported.

3.1.2 Execution

An overview of the execution phase is illustrated in Figure 3.3. The process begins at **hs_scan**. Rose determines three execution scenarios: (1) all patterns are literals, (2) no literals wake up FAs, and (3) literals wake up FAs. The last scenario is the most common. Its flow reaches **roseBlockExec**, where one of the string matchers is called through **hwlmExec** together with a **roseCallback** function. The **roseCallback** function eventually calls **roseRunProgram**. This is where the **left** or **suffix** FAs are checked, woken up, and executed.

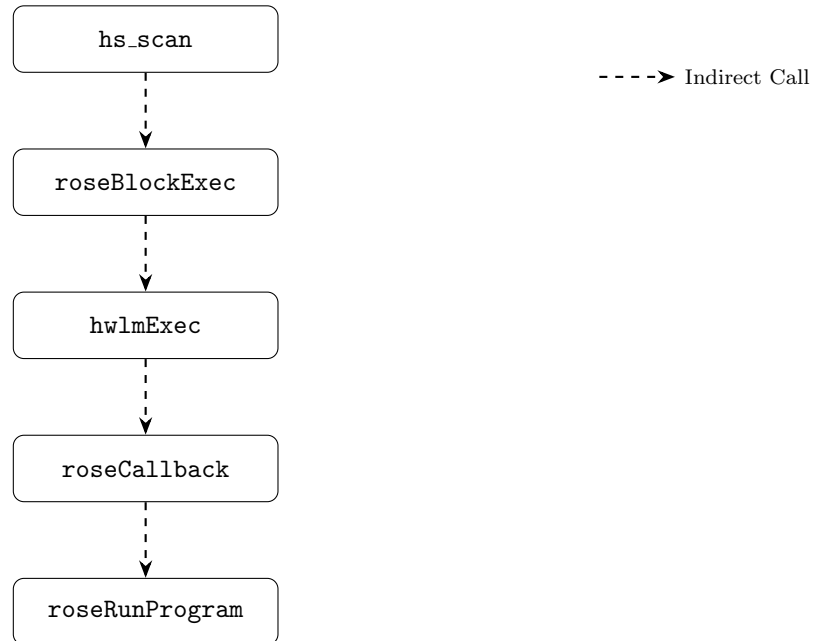


Figure 3.3: Dependency graph of main functions in the execution phase.

3.2 The FDR string matcher

There are at least four SIMD-based string matchers in Hyperscan: Noodle is used for a single literal, Teddy [13] or Harry [18] for 2 to 300 literals, and FDR for more than 300 literals. The last three matchers are prefilter-based: they first filter the text to find candidate matches, and then verify them. FDR is our main candidate for analysis. It extends the classical Shift-Or [1] algorithm to match multiple patterns at once. In this section, we review the Shift-Or algorithm and the two FDR phases, compilation and execution.

3.2.1 The Shift-Or algorithm

Let Σ be the alphabet, T be the text, and P be the pattern. Assume a machine word size of w bits, which restricts the pattern length to $|P| \leq w$. Characters in P and T are zero-indexed from left to right. Conversely, register bits are zero-indexed from right to left, starting with the least significant bit (LSB) at position 0.

During the preprocessing stage, we construct a bitmask for each character in the alphabet, denoted by a function $M : \Sigma \rightarrow \{0, 1\}^w$. For each character $c \in \Sigma$, the mask is defined as:

$$M[c][i] = \begin{cases} 0 & \text{if } i < |P| \text{ and } P[i] = c \\ 1 & \text{otherwise, for } i \in [0, w-1] \end{cases} \quad (3.1)$$

The search phase proceeds by initializing a state register S to all 1s ($S \leftarrow \sim 0$). At each step $i \geq 0$, the register S is updated using the recurrence:

$$S \leftarrow (S \ll 1) \mid M[T[i]]. \quad (3.2)$$

The algorithm operates by shifting the state register left by one position, introducing a 0 at the LSB. This 0 propagates to the left as long as the text characters match the pattern characters. If a mismatch occurs, the 0 is overridden by a 1 from the character mask, halting the propagation. Consequently, the state register S tracks the longest prefix of P that matches a suffix of T ending at position i . If $S[|P|-1] = 0$ at step i , a match ends at position i in the text. Correctness is established via induction on the invariant: at step i ,

$$S[j] = 0 \iff T[i-j \dots i] = P[0 \dots j], \quad \forall j \in [0, |P|-1]. \quad (3.3)$$

Example 3.1. Let the pattern be $P = \text{"abac"}$ ($|P| = 4$) and the text be $T = \text{"bbabac"}$. Assume $w \geq 4$; for clarity, we only display the lowest 4 bits. First, we construct the masks M :

- $M[a] = 1010_2$ (since $P[0]$ and $P[2]$ are 'a')
- $M[b] = 1101_2$ (since $P[1]$ is 'b')
- $M[c] = 0111_2$ (since $P[3]$ is 'c')
- $M[x] = 1111_2$ for any other character $x \notin P$

We initialize $S = 1111_2$. As we read T , we update $S \leftarrow (S \ll 1) \mid M[T[i]]$:

- $i = 0, T[0] = b$: $S \leftarrow 1110_2 \mid 1101_2 = 1111_2$
- $i = 1, T[1] = b$: $S \leftarrow 1110_2 \mid 1101_2 = 1111_2$
- $i = 2, T[2] = a$: $S \leftarrow 1110_2 \mid 1010_2 = 1110_2$ (Match $P[0]$)

3.2 The FDR string matcher

- $i = 3, T[3] = b$: $S \leftarrow 1100_2 \mid 1101_2 = 1101_2$ (Match $P[0 \dots 1]$)
- $i = 4, T[4] = a$: $S \leftarrow 1010_2 \mid 1010_2 = 1010_2$ (Match $P[0 \dots 2]$, and new $P[0]$)
- $i = 5, T[5] = c$: $S \leftarrow 0100_2 \mid 0111_2 = 0111_2$ (Match $P[0 \dots 3]$)

At $i = 5$, the bit at index $|P| - 1 = 3$ is 0 ($S[3] = 0$), indicating a full pattern match ending at position 5. ■

3.2.2 Compilation

First, the literals are grouped into *buckets*. Each bucket can be regarded as an indeterminate string (see e.g., [8]).

Definition 3.2. Let Σ be an alphabet. An indeterminate string $\mathbf{P}$ of length n over Σ is a sequence of n non-empty sets of characters from Σ , i.e.,

$$\mathbf{P} = p_1 p_2 \dots p_n,$$

where $\emptyset \neq p_i \subseteq \Sigma$ for all $i \in [1, n]$. A position equal to Σ is usually referred to as a *don't care* position. A string T over Σ matches the indeterminate string $\mathbf{P}$ if and only if $|T| = n$ and $T[i] \in p_i$ for all $i \in [1, n]$. ■

The set of indeterminate strings over Σ is a strict subset of the set of star-free regular expressions over Σ . For example, the regular expression $ab|cd$ is not an indeterminate string. Let $\{P_1, P_2, \dots, P_k\}$ be the set of literals in a bucket. Let p_{ij} be the j -th character of literal P_i . The literals are right-aligned and padded with the don't care character $*$ to form an indeterminate string $\mathbf{P}$ of length $n = \max\{|P_1|, |P_2|, \dots, |P_k|\}$, i.e., $\mathbf{P} = p_1 p_2 \dots p_n$, where

$$p_{n-j} = \begin{cases} \{p_{i, |P_i| - j} \mid 1 \leq i \leq k, j \leq |P_i|\}, & \text{if } p_{i, |P_i| - j} \text{ is defined for every } i, \\ \Sigma, & \text{otherwise.} \end{cases}$$

The implementation of FDR uses the following conventions. Each character has 8 bits. There are 8 buckets $B_0, \dots, B_7$. Each register has 128 bits. We access the bit indices in a register and the character positions in a string from right to left.

Let $\mathbf{P}$ be a set of patterns. Let $\tilde{\mathbf{P}} = (P_1, \dots, P_m)$ be a permutation of $\mathbf{P}$ where the patterns are sorted by length, then by lexicographical order. The assignment function is derived from a partition of $\tilde{\mathbf{P}}$ into 8 buckets, defined by the indices $0 = b_0 < b_1 < \dots < b_6 < b_7 < b_8 = m$, that is, $P_{b_j}, \dots, P_{b_{j+1}-1}$ belong to B_j . It aims to minimize the cost function

$$f(b_1, \dots, b_6) = \sum_{j=0}^7 \frac{(b_{j+1} - b_j)^\alpha}{|P_{b_j}|^\beta}. \quad (3.4)$$

The implementation chooses $\alpha = 1.05$ and $\beta = 3$. If one uses the same cost function over the set of all possible assignments, the optimal assignment can be achieved by partitioning $\tilde{\mathbf{P}}$. Thus FDR effectively reduces a combinatorial optimization problem to a dynamic programming problem. We state this fact as follows.

Proposition 3.1 Define the cost function g over the set of assignment functions as

$$g(\sigma) = \sum_{j \in B} \frac{|\sigma^{-1}(j)|^\alpha}{\min_{p \in \sigma^{-1}(j)} |p|^\beta}.$$

Then $\min_{b_1, \dots, b_6} f(b_1, \dots, b_6) = \min_{\sigma} g(\sigma)$.

3.2 The FDR string matcher

Proof. The fact that $\min g(\sigma) \leq \min f(b_1, \dots, b_6)$ is straightforward, since any partition of $\tilde{\mathbf{P}}$ into 8 buckets defines an assignment function.

We prove the other direction. Let σ be an assignment function that minimizes g , and suppose that its 8 buckets have sizes $n_0, n_1, \dots, n_7$ with $\sum_{i=0}^7 n_i = m$. Let $L_j = \min_{p \in \sigma^{-1}(j)} |p|$ be the minimum pattern length in bucket j . Without loss of generality, we can assume that $L_0 \leq L_1 \leq \dots \leq L_7$. Construct a new, contiguous partition of the sorted array $\tilde{\mathbf{P}}$ using these same sizes $n_0, \dots, n_7$. The boundaries are $b_0 = 0$ and $b_{j+1} = b_j + n_j$. Let $L'_j = \lfloor p_{b_j} \rfloor$ be the minimum pattern length in bucket j under the partition. Then $L'_j \geq L_j$ for all j , and therefore

$$\frac{n_j^\alpha}{(L'_j)^\beta} \leq \frac{n_j^\alpha}{L_j^\beta}, \forall j \in \{0, \dots, 7\}.$$

This yields a new assignment function σ' such that $g(\sigma') \leq g(\sigma)$. Since σ' is a partition of $\tilde{\mathbf{P}}$, we have $\min f(b_1, \dots, b_6) \leq g(\sigma')$. This completes the proof. $\square$

After assigning the patterns to buckets, the algorithm builds a mask for each character. The mask for a character c is a bitmask of size 128 where the left-most 64 bits are set to 0. In the right-most 64 bits, the $(8i + j)$ -th bit ($0 \leq i, j \leq 7$) is set to 0 if either the character c appears in position i (indexing from the right) of a pattern in bucket B_j , or the bucket B_j is not empty and patterns in B_j have length less than or equal to i . In this case it is called a *padding bit*. Otherwise, the bit is set to 1.

For example, consider the buckets $[[ab, bc], [abc], [], [], [], [], [], []]$. Let x denote any character other than a, b, c . The masks are as follows, where the padding bits are in bold.

$$\begin{aligned} M[a] &= \underbrace{00 \dots 00}_{64} \text{ 1111100 1111100 1111100 1111100 1111100 1111100 1111110 1111111} \\ M[b] &= \underbrace{00 \dots 00}_{64} \text{ 1111100 1111100 1111100 1111100 1111100 1111110 1111100 1111110} \\ M[c] &= \underbrace{00 \dots 00}_{64} \text{ 1111100 1111100 1111100 1111100 1111100 1111110 1111111 1111101} \\ M[x] &= \underbrace{00 \dots 00}_{64} \text{ 1111100 1111100 1111100 1111100 1111100 1111110 1111111 1111111.} \end{aligned}$$

The last step is to build a hash table for the patterns. Let h be a hash function. Investigating it in the original implementation is left to future work. An array `arr` is built such that `arr[h(p)]` stores information about the bucket containing the pattern p .

3.2.3 Execution

The compiled data structure is used to scan a text. A state mask R is maintained during the process. Each iteration processes 8 bytes of the text. At iteration `ITER`, the $(8i + j)$ -th bit ($0 \leq i \leq 15, 0 \leq j \leq 7$) is 0 if there is currently *no contradiction* to a match of a pattern in bucket B_j ending at position $8 \times \text{ITER} + i$. If bucket B_j has patterns of length ℓ_j , then there cannot be a match in this bucket at a position smaller than $\ell_j - 1$. Therefore, we initialize the state mask to all 0's, except for bits $8i + j$ where $0 \leq i < \ell_j - 1$. After an iteration, we update $R := R \gg 64$ and continue matching the next 8 bytes. This ensures that, at each position, the prefix of length at most 8 is considered. Returning to the example, the initial state mask is

$$R \leftarrow \underbrace{00 \dots 00}_{104} \text{ 11111100 11111110 11111111.}$$

3.2 The FDR string matcher

We now match against the text *abac*. Since the text has only 4 characters, there is only one iteration. We detail the scan character by character, considering the right-most 4 bytes. The new bit set to 1 after each character is scanned is underlined.

Scan	Update	Result
<i>b</i>	$R \leftarrow R[(M[b] \ll 0)]$	... 11111100 111111 <u>1</u> 0 11111110 11111111
<i>a</i>	$R \leftarrow R[(M[a] \ll 1)]$	... 11111100 11111110 1111111 <u>1</u> 11111111
<i>b</i>	$R \leftarrow R[(M[b] \ll 2)]$	... 11111100 11111110 11111110 11111111
<i>c</i>	$R \leftarrow R[(M[c] \ll 3)]$	... 1111110 <u>1</u> 11111110 11111110 11111111

The algorithm reports a match in bucket B_0 ending at position 1, a match in bucket B_1 ending at position 2, and a match in bucket B_0 ending at position 3. Since the pattern lengths in each bucket are known, we can retrieve the beginning positions of these potential matches and verify them by exact string comparison. In this case, all three potential matches are indeed matches. The pseudocode is given in Algorithm 1 and Algorithm 2.

Algorithm 1 Compilation phase: building masks

```

1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3: for  $c = 00000000$  to  $11111111$  do
4:    $M[c] \leftarrow \underbrace{00\dots00}_{64} \underbrace{11\dots11}_{64}$  ▷ Initialize.
5:   for  $j = 0$  to  $7$  and  $B_j$  is not empty do
6:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
7:     for  $i = \ell_j$  to  $7$  do
8:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
9:     end for
10:  end for
11: end for
12: for  $j = 0$  to  $7$  do
13:   for each pattern  $x$  in bucket  $B_j$  do
14:     for  $i = 0$  to  $\text{length}(x) - 1$  do
15:        $M[x[i]][8i + j] \leftarrow 0$ 
16:     end for
17:   end for
18: end for

```

Algorithm 2 Execution phase: scanning the text

```

1:  $R \leftarrow$  a bitmask of size 128 where the  $(8i + j)$ -th bit is 0 if  $i \geq \ell_j - 1$  and 1 otherwise.
2:  $T \leftarrow$  the text to be scanned.
3: for ITER = 0 to  $\lfloor \frac{\text{length}(T)}{8} \rfloor - 1$  do
4:    $V \leftarrow T[8 \times \text{ITER} \dots 8 \times \text{ITER} + 7]$ 
5:   for  $j = 0$  to  $7$  do
6:      $R \leftarrow R[(M[V[j]] \ll j)]$ 
7:   end for
8:   Report potential matches at positions where the corresponding bits in  $R$  are 0. Verify those
   potential matches by hashing then by exact string comparison.
9:    $R \leftarrow R \gg 64$ 
10: end for

```

If the text contains *ac*, a match at position 1 for bucket B_0 is still reported. *Super-characters* are introduced. Let s be the number of bits in a super-character. For example, instead of assigning $M[a]$ as in the first step of the compilation phase, we build a mask for the super-character

$$(b[0 \dots s - 8] \ll 8) | a.$$

3.2 The FDR string matcher

If a character is at the end of a pattern, for example b in the pattern ab , then we assign the mask for

$$0 \ll 8 | b.$$

However, the algorithm may then *fail* to report the match of ab in bucket B_0 ending at position 2 for the text $babc$. The reason is that when scanning the second b , it queries and ORs the mask for the super-character $(b[0 \dots s-8] \ll 8) | c$ instead of the mask for $0 \ll 8 | b$. The idea is to set the bits that may introduce a wrong contradiction to 0 in the mask for the super-character beforehand. More precisely, if a character c appears at position 0 of a pattern in bucket B_j , then we set the bit at position j to 0 in the mask for the super-character

$$(x[0 \dots s-8] \ll 8) | c$$

for *every* character x . This can be done more efficiently, since there are only 2^{s-8} such super-characters. The corrected compilation pseudocode is given in Algorithm 3. The changes and additions are highlighted in blue.

Algorithm 3 Compilation phase: building masks with super-characters

```

1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3:  $s \leftarrow$  the number of bits in a super-character.
4: for  $c = \underbrace{00 \dots 00}_s$  to  $\underbrace{11 \dots 11}_s$  do
5:    $M[c] \leftarrow \underbrace{00 \dots 00}_{64} \underbrace{11 \dots 11}_{64}$  ▷ Initialize.
6:   for  $j = 0$  to 7 and  $B_j$  is not empty do
7:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
8:     for  $i = \ell_j$  to 7 do
9:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
10:    end for
11:  end for
12: end for
13: for  $j = 0$  to 7 do
14:   for each pattern  $x$  in bucket  $B_j$  do
15:      $f \leftarrow x[0]$  ▷ Handle the end character.
16:     for  $y = \underbrace{0 \dots 0}_{s-8}$  to  $\underbrace{1 \dots 1}_{s-8}$  do
17:        $M[(y \ll 8) | f][8j] \leftarrow 0$ 
18:     end for
19:     for  $i = 1$  to  $\text{length}(x) - 1$  do
20:        $M[x[i]][8i + j] \leftarrow 0$ 
21:     end for
22:   end for
23: end for

```

A potential piece of text is hashed by the function h in the compilation phase and checked against the corresponding bucket. If there is a match, the beginning position of the pattern is retrieved and verified by exact string comparison.

Another parameter is the *stride*. For example, if the stride is 2, then we traverse j at 0, 2, 4, and 6. Each step of j requires a query to the mask dictionary, which is not constant-time in the implementation. The trade-off is that there are more positives to verify.

The simplified FDR model above was also used as an experimental object. In particular, we implemented variants that isolate the effects of bucket assignment, super-characters, and stride choices on the probability of reporting positives. These experiments suggest that a complete analysis should not only count SIMD operations, but also account for the verification cost created by false positives.

3.3 Related problems

3.3.1 Optimizing regular expression decomposition

Both representations in Figure 3.2 are likely suboptimal for theoretical analysis. First, the sub-FA may appear several times, even though it appears only once in the regular expression, for example in the case of $(abc|def)[0-9]^+xyz$. Second, although we had to mark xyz as a terminal vertex in Figure 3.2, this is not strictly accurate: the matching process terminates at the end of the sub-FA, not at the end of xyz . If there are remaining FAs or literals to the right, it is the sub-FA for $[0-9]^+$ that will determine the next matching position. Therefore, we propose a more convenient representation as follows. Each vertex is either a literal or a sub-FA. There are three types of directed edges.

- Edges that use the *end* positions of the matches of the source vertex to determine the *start* positions of the matches of the target vertex. They connect two literals or a literal and a sub-FA. We call this type of edge **setStart**.
- Edges that use the *start* positions of the matches of the source vertex to determine the *end* positions of the matches of the target vertex. They connect a literal and a sub-FA. We call this type of edge **setEnd**.
- Edges that act as pure triggers, without any position information. They connect two sub-FAs. We call this type of edge **trigger**.

Consider another example of $abc[0-9]^+xyz(de)^*$. Its dependency graph in the new representation is shown in Figure 3.4.

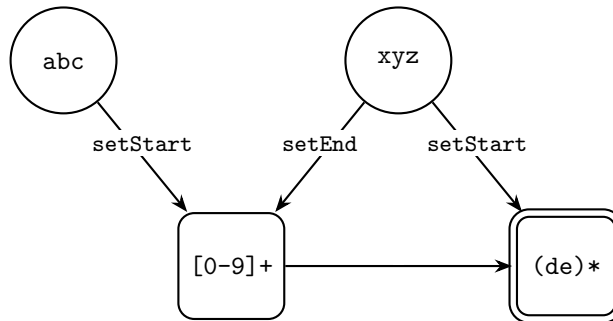


Figure 3.4: Proposed sequential dependency graph representation for $abc[0-9]^+xyz(de)^*$.

3.3.2 Analysis of the FDR string matcher

A complete analysis of the FDR string matcher would take all of its ingredients into account: the assignment algorithm, the super-character size, the stride, the hash function, and the verification cost. We leave this for future work.

3.3.3 Statistics of regular expressions

One tractable part of analyzing Hyperscan is to study statistics of regular expressions under a distribution. We use the BST-like distribution from Definition 2.3. In this model, u is the probability that an internal node is a concatenation, v is the probability that it is an alternation, and $1 - u - v$ is the probability that it is a Kleene star. These parameters can be fitted to concrete pattern sets. For

3.3 Related problems

instance, maximum-likelihood fitting on the Snort rule set gives $u \approx 0.9$ and $v \approx 0.09$, as discussed in the previous chapter.

Since Hyperscan extracts and matches literals before executing the sub-FAs, it is worth considering the average length of the longest literal that Hyperscan attempts to match. Let this quantity be denoted by a random variable Y_n for a random regular expression tree of size n . If the root is a concatenation, then literals from the left and right subtrees can be concatenated, so the longest literal is the sum of the two subtree contributions. If the root is an alternation, then a string follows one branch or the other, so the longest literal is the maximum of the two subtree contributions. If the root is a Kleene star, the language contains the empty string; in the model used here, Hyperscan does not extract a literal through that root, and the contribution is 0.

$$Y_{n+2} \stackrel{d}{=} I \left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} \right) + J \max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right), n \geq 1, \quad Y_1 = 1, Y_2 = 0. \quad (3.5)$$

The recurrence is obtained by conditioning on the root operator. For a tree of size $n + 2$, the root consumes one unit of size and the two subtrees have total size $n + 1$. The split is chosen uniformly, hence the random variable U_n is uniformly distributed on $\{1, \dots, n\}$ and the two subtree sizes are U_n and $n + 1 - U_n$. Superscripts denote mutually independent copies. The variables I and J indicate whether the root is a concatenation or an alternation; they satisfy $\mathbb{E}[I] = u$, $\mathbb{E}[J] = v$, and $I + J \leq 1$. When both indicators are zero, the root is a Kleene star and the contribution is 0. The base cases are $Y_1 = 1$ for a single letter and $Y_2 = 0$ for a single Kleene-star root. A detailed analysis of the asymptotic behavior of $\mathbb{E}[Y_n]$ is given in the next chapter.

4

The Length of Longest Literals

Recall that in the BST-like model, u is the probability of a concatenation node, v is the probability of an alternation node, and $1 - u - v$ is the probability of a Kleene star node. For a random regular expression tree of size n , the random variable Y_n denotes the length of the longest literal extracted by Hyperscan, and we write $y_n = \mathbb{E}[Y_n]$. This chapter studies the asymptotic behavior of (y_n) .

We first bound (y_n) between deterministic recurrences. The upper bound replaces every maximum by a sum, while the lower bound uses the maximum of the expectations. The main deterministic object is the tightened lower bound (z_n) defined in (4.4). We prove successively that $z_n = n^{\alpha+o(1)}$, that $z_n = \Theta(n^\alpha)$, and, under the stronger condition $u > 1/2$, that $z_n \sim Cn^\alpha$ for some $C > 0$; see Theorem 4.3. The last section returns to the original stochastic recurrence and outlines a fixed-point approach for the asymptotics of (y_n) .

4.1 Lower and upper bounds

Consider the recurrence (3.5). Denote $y_n := \mathbb{E}[Y_n]$ for $n \geq 1$. Taking expectations in (3.5) gives, for $n \geq 1$,

$$\begin{aligned} y_{n+2} &= \frac{u}{n} \sum_{i=1}^n (y_i + y_{n+1-i}) + \frac{v}{n} \sum_{i=1}^n \mathbb{E} \left[\max \left(Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right] \\ &= \frac{2u}{n} \sum_{i=1}^n y_i + \frac{v}{n} \sum_{i=1}^n \mathbb{E} \left[\max \left(Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right]. \end{aligned} \tag{4.1}$$

Using the inequality $\max(a, b) \leq a + b$ for $a, b \geq 0$, we have

$$y_{n+2} \leq \frac{2(u+v)}{n} \sum_{i=1}^n y_i.$$

Since the right-hand side of the inequality is an increasing function of $(y_1, \dots, y_n)$, the sequence (y_n) is bounded above by the solution to the recurrence

$$z_{n+2} = \frac{2(u+v)}{n} \sum_{i=1}^n z_i, \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \tag{4.2}$$

4.2 Asymptotics of the lower bound recurrence

Similarly, the inequality $\max(a, b) \geq \frac{1}{2}(a + b)$ gives a lower bound sequence

$$z_{n+2} = \frac{2u + v}{n} \sum_{i=1}^n z_i, \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (4.3)$$

A transfer theorem from analytic combinatorics gives the asymptotic behavior of (4.2) and (4.3); see Appendix A.1.1. The lower bound can be tightened by using

$$\mathbb{E} \left[\max \left(Y_i^{(1)}, Y_{n+1-i}^{(2)} \right) \right] \geq \max \left(\mathbb{E} \left[Y_i^{(1)} \right], \mathbb{E} \left[Y_{n+1-i}^{(2)} \right] \right) = \max(y_i, y_{n+1-i}).$$

This is not sharp since equality holds only when $Y_i^{(1)} = Y_{n+1-i}^{(2)}$ a.s. The tightened lower bound recurrence is then given by

$$z_{n+2} = \frac{2u}{n} \sum_{i=1}^n z_i + \frac{v}{n} \sum_{i=1}^n \max(z_i, z_{n+1-i}), \quad n \geq 1, \quad z_1 = 1, z_2 = 0. \quad (4.4)$$

We analyze it in more detail in the next section.

4.2 Asymptotics of the lower bound recurrence

The sequence defined by (4.4) is bounded between those defined by (4.2) and (4.3). Under the conditions

$$u, v \geq 0, \quad u < 1, \quad u + v \leq 1, \quad 2u + v > 1, \quad (4.5)$$

both (4.2) and (4.3) grow as a pure power of n , without any lower-order factor such as a logarithm. In this section, we use several techniques to prove asymptotic theorems of increasing strength. The strongest theorem gives exact asymptotics, namely $z_n \sim Cn^\alpha$ for some constants $C > 0$ and $\alpha > 0$.

As a heuristic, assume that $z_n \sim Cn^\alpha$ for some constants $C > 0$ and $\alpha > 0$. Substituting this estimate into (4.4), we get

$$\begin{aligned} Cn^\alpha &\sim \frac{2u}{n} \sum_{i=1}^n Ci^\alpha + \frac{v}{n} \sum_{i=1}^n C \max(i^\alpha, (n+1-i)^\alpha) \\ &\sim C \left(\frac{2u}{n} \int_0^n Cx^\alpha dx + \frac{v}{n} \int_0^n C \max(x^\alpha, (n-x)^\alpha) dx \right) \\ &= C \left(\frac{2u}{n} \int_0^n x^\alpha dx + \frac{2v}{n} \cdot \int_{n/2}^n x^\alpha dx \right) \\ &= C \left(\frac{2u}{n} \cdot \frac{n^{\alpha+1}}{\alpha+1} + \frac{2v}{n} \cdot \frac{n^{\alpha+1} - (n/2)^{\alpha+1}}{\alpha+1} \right) \\ &= \frac{C}{\alpha+1} (2(u+v) - v2^{-\alpha}) n^\alpha. \end{aligned}$$

Thus, we must have

$$\alpha + v2^{-\alpha} = 2(u+v) - 1. \quad (4.6)$$

The condition $2u + v > 1$ ensures that $\alpha > 0$; otherwise $2u + v = 1$. Numerical experiments regarding the ratio z_n/n^α are illustrated in Figure 4.1. We now state the asymptotic theorems.

Theorem 4.1 *Let α be the solution to (4.6). Then, $z_n = n^{\alpha+o(1)}$.*

Theorem 4.2 *Let α be the solution to (4.6). Then, $z_n = \Theta(n^\alpha)$.*

4.2 Asymptotics of the lower bound recurrence

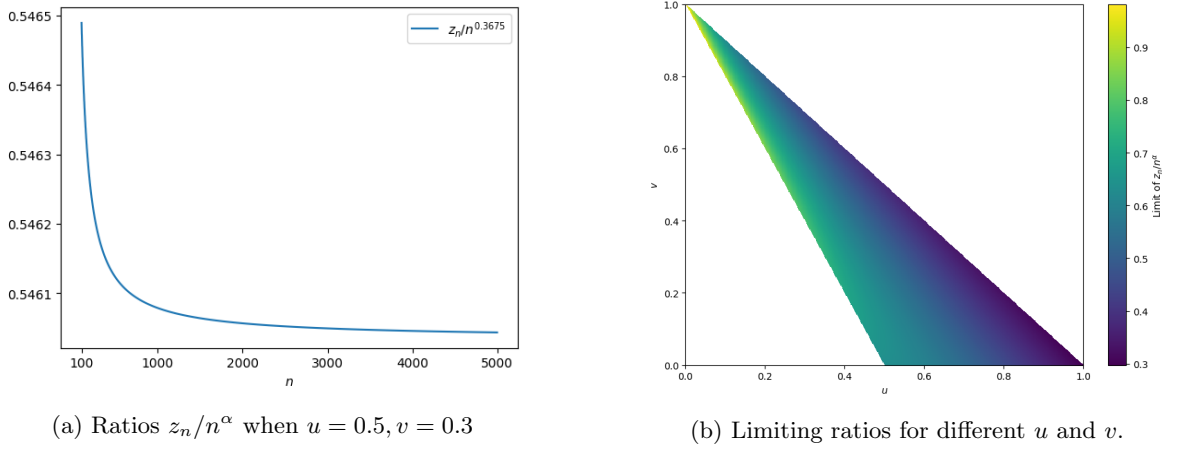


Figure 4.1: Computation of z_n/n^α

For the exact asymptotics, we need to impose stricter conditions, for reasons that will become clear later.

$$u, v \geq 0, \quad \frac{1}{2} < u < 1, \quad u + v \leq 1, \quad (4.7)$$

Theorem 4.3 *Let (z_n) be defined by (4.4), let α be the solution to (4.6), and let u, v be the parameters satisfying (4.7). Then $z_n \sim Cn^\alpha$ for some constant $C > 0$.*

We begin with Theorem 4.1. Rewrite the recurrence in (4.4) as

$$z_{n+2} = \frac{1}{n} \sum_{i=1}^n F(z_i, z_{n+1-i}), \quad n \geq 1, \quad (4.8)$$

where

$$F(x, y) = u(x + y) + v \max(x, y), \quad x, y > 0. \quad (4.9)$$

Keeping in mind the heuristic $z_n \sim Cn^\alpha$, we try to approximate the right-hand side of (4.8).

Lemma 4.4 *For every $\alpha \in (0, 1]$, we have*

$$\frac{1}{n} \sum_{i=1}^n F(i^\alpha, (n+1-i)^\alpha) \sim n^\alpha \Lambda(\alpha),$$

$$\text{where } \Lambda(\alpha) = \int_0^1 F(x^\alpha, (1-x)^\alpha) dx = \frac{2(u+v) - v2^{-\alpha}}{\alpha+1}.$$

Proof. We will prove the equivalent limit

$$\frac{1}{n} \sum_{i=1}^n F\left(\left(\frac{i}{n}\right)^\alpha, \left(1 - \frac{i}{n} + \frac{1}{n}\right)^\alpha\right) \rightarrow \Lambda(\alpha).$$

Note that Λ is continuous on $[0, 1]$ with respect to α , and the left-hand side is different from its Riemann sum only by a perturbation $1/n$ in the second argument. We show that the perturbation is negligible. Define for $x \in [0, 1]$ the functions

$$f_n(x) = F\left(x^\alpha, \left(1 - x + \frac{1}{n}\right)^\alpha\right) \text{ and } f(x) = F(x^\alpha, (1-x)^\alpha).$$

4.2 Asymptotics of the lower bound recurrence

For $a, b, c \geq 0$ and $\alpha \in (0, 1]$, we have

$$|\max(a, b) - \max(a, c)| \leq |b - c| \text{ and } (a + b)^\alpha \leq a^\alpha + b^\alpha.$$

Hence,

$$\|f_n - f\|_\infty \leq \sup_{x \in [0, 1]} (u + v) \left(\left(1 - x + \frac{1}{n}\right)^\alpha - (1 - x)^\alpha \right) \leq (u + v) \frac{1}{n^\alpha} \rightarrow 0.$$

Therefore,

$$\left| \frac{1}{n} \sum_{i=1}^n f_n \left(\frac{i}{n} \right) - \frac{1}{n} \sum_{i=1}^n f \left(\frac{i}{n} \right) \right| \leq \|f_n - f\|_\infty \rightarrow 0.$$

□

The unique value of α such that $\Lambda(\alpha) = 1$ is the solution to (4.6). To see this, compute

$$\begin{aligned} \Lambda'(\alpha) &= \frac{v2^{-\alpha} \ln 2}{\alpha + 1} - \frac{2(u + v) - v2^{-\alpha}}{(\alpha + 1)^2} = \frac{v2^{-\alpha}((\alpha + 1) \ln 2 + 1) - 2(u + v)}{(\alpha + 1)^2}, \\ (2^{-\alpha}((\alpha + 1) \ln 2 + 1))' &= -2^{-\alpha}(\alpha + 1) \ln^2(2) < 0. \end{aligned}$$

This implies

$$(\alpha + 1)^2 \Lambda'(\alpha) < (\alpha + 1)^2 \Lambda'(0) = -(2u + (1 - \ln 2)v) < 0.$$

Hence, Λ is strictly decreasing. We also have $\Lambda(0) = 2u + v > 1$ and $\Lambda(1) = u + v/2 < 1$. We exploit the fact that $\Lambda(\beta) > 1$ for $\beta < \alpha$ and $\Lambda(\beta) < 1$ for $\beta > \alpha$ to prove Theorem 4.1.

Proof of Theorem 4.1. Let $A_n(\beta) = \frac{1}{n} \sum_{i=1}^n F(i^\beta, (n + 1 - i)^\beta)$. For the upper bound, let $\beta \in (\alpha, 1]$.

Then $\Lambda(\beta) < 1$. Hence there exists N such that for all $n \geq N$,

$$A_n(\beta) < n^\beta < (n + 2)^\beta.$$

Let $M_n = \max_{1 \leq k \leq n} \frac{z_i}{k^\beta}$. For $n \geq N$, we have

$$z_{n+2} = \frac{1}{n} \sum_{i=1}^n F(z_i, z_{n+1-i}) \leq M_n A_n(\beta) < M_n (n + 2)^\beta.$$

Hence, $\frac{z_{n+2}}{(n + 2)^\beta} < M_n$. Thus (M_n) is eventually decreasing, and $z_n = O(n^\beta)$.

A similar argument applies for the lower bound, with an adjustment to deal with z_2 . We have

$$A_n(\beta) \sim n^\beta \sim (n + 2)^\beta \text{ and } A_n(\beta) \sim \frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n + 1 - i)^\beta).$$

Assume that $\beta < \alpha$. Then, $\Lambda(\beta) > 1$. Hence there exists $N \geq 3$ such that for all $n \geq N$,

$$\frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n + 1 - i)^\beta) > (n + 2)^\beta.$$

Let $m_n = \min_{3 \leq i \leq n-2} \frac{z_i}{i^\beta}$. For $n \geq N$, we have

$$z_{n+2} \geq \frac{1}{n} \sum_{i=3}^{n-2} F(z_i, z_{n+1-i}) \geq m_n \frac{1}{n} \sum_{i=3}^{n-2} F(i^\beta, (n + 1 - i)^\beta) > m_n (n + 2)^\beta.$$

Hence, $\frac{z_{n+2}}{(n + 2)^\beta} > m_n$. Thus (m_n) is eventually increasing, and $z_n = \Omega(n^\beta)$. □

4.2 Asymptotics of the lower bound recurrence

The proof of Theorem 4.2 uses a fine-grained induction. We first prepare integral bounds for the sum of the maximum terms.

Lemma 4.5 *Let $f : [0, \infty) \rightarrow \mathbb{R}$ be an increasing function. For every integer $n \geq 1$, we have*

$$S_1 := \sum_{i=0}^{n-1} \max(f(i), f(n-1-i)) \leq 2 \int_{n/2}^n f(x) dx \leq S_2 := \sum_{i=1}^n \max(f(i), f(n+1-i)).$$

Proof. We first prove the upper bound on the first sum. We will use the following bounds for $t \geq 0$:

$$f(t) \leq \int_t^{t+1} f(x) dx \leq 2 \int_{t+1/2}^{t+1} f(x) dx.$$

If $n = 2k$, we have

$$S_1 = 2 \sum_{i=k}^{2k-1} f(i) \leq 2 \sum_{i=k}^{2k-1} \int_i^{i+1} f(x) dx.$$

If $n = 2k + 1$, we have

$$S_1 = 2 \sum_{i=k+1}^{2k} f(i) + f(k) \leq 2 \sum_{i=k+1}^{2k} \int_i^{i+1} f(x) dx + 2 \int_{k+1/2}^{k+1} f(x) dx.$$

The lower bound on the second sum follows analogously from the following estimates. For $t \geq 1$, we have

$$f(t) \geq \int_{t-1}^t f(x) dx \geq 2 \int_{t-1}^{t-1/2} f(x) dx.$$

□

Proof of Theorem 4.2. We prove the lower bound. First, $z_n > 0$ for all $n \neq 2$. The main idea is to isolate z_2 . The induction hypothesis is $z_n \geq K(n+L)^\alpha$ for $n \neq 2$. The base cases and the exact constants are determined later. We need

$$\max(z_2, z_{n-1}) = z_{n-1} \geq \max((2+L)^\alpha, (n-1+L)^\alpha).$$

Thus, for $n \geq 3$,

$$\begin{aligned} nz_{n+2} &= 2u \sum_{i=1}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}) \\ &\geq K \left(2u \sum_{i=1}^n (i+L)^\alpha + v \sum_{i=1}^n \max((i+L)^\alpha, (n+1-i+L)^\alpha) \right) - K2u(2+L)^\alpha \\ &\geq K \left(2u \int_0^n (x+L)^\alpha dx + 2v \int_{n/2}^n (x+L)^\alpha dx \right) - K2u(2+L)^\alpha. \end{aligned}$$

Let

$$\begin{aligned} A_n &= 2u \int_0^n (x+L)^\alpha dx + 2v \int_{n/2}^n (x+L)^\alpha dx \\ &= \frac{1}{\alpha+1} \left((2u+2v)(n+L)^{\alpha+1} - 2v \left(\frac{n}{2} + L \right)^{\alpha+1} \right). \end{aligned}$$

Using the expansions

$$\begin{aligned} (n+L)^{\alpha+1} &= n^{\alpha+1} + (\alpha+1)Ln^\alpha + O(n^{\alpha-1}), \\ \left(\frac{n}{2} + L \right)^{\alpha+1} &= 2^{-\alpha+1}n^{\alpha+1} + (\alpha+1)L2^{-\alpha}n^\alpha + O(n^{\alpha-1}), \end{aligned}$$

4.2 Asymptotics of the lower bound recurrence

we get

$$\begin{aligned} A_n &= n^{\alpha+1} + L(2u + 2v - 2v2^{-\alpha})n^\alpha + O(n^{\alpha-1}) \\ &= n^{\alpha+1} + L(\alpha + 1 - v2^{-\alpha})n^\alpha + O(n^{\alpha-1}). \end{aligned}$$

On the other hand,

$$n(n + L + 2)^\alpha = n^{\alpha+1} + \alpha(L + 2)n^\alpha + O(n^{\alpha-1}).$$

Hence,

$$A_n - 2u(2 + L)^\alpha - n(n + L + 2)^\alpha = (L(1 - v2^{-\alpha}) - 2\alpha)n^\alpha + O(n^{\alpha-1}).$$

Since $1 - v2^{-\alpha} > 0$, choose $L > \frac{2\alpha}{1 - v2^{-\alpha}}$, then there exists N such that for all $n \geq N$,

$$A_n - 2u(2 + L)^\alpha - n(n + L + 2)^\alpha \geq 0,$$

implying that $z_{n+2} \geq K(n + L + 2)^\alpha$. Finally, we choose

$$K = \min_{\substack{1 \leq i \leq N+1 \\ i \neq 2}} \frac{z_i}{(i + L)^\alpha} > 0$$

to satisfy the base cases $i \leq N + 1$, excluding $i = 2$ where $z_2 = 0$.

We now prove the upper bound. Our induction hypothesis is $z_n \leq \tilde{K}(n - 1)^\alpha$ for $n \geq 2$, for some constant $\tilde{K} > 1$. The base case $z_2 = 0$ is trivially satisfied. For $n \geq 2$, we have

$$\begin{aligned} nz_{n+2} &= 2u + 2u \sum_{i=2}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}) \\ &\leq \tilde{K} \left(2u \sum_{i=1}^{n-1} i^\alpha + v \sum_{i=0}^{n-1} \max(i^\alpha, (n-1-i)^\alpha) \right) + 2u \\ &\leq \tilde{K} \left(2u \int_1^n x^\alpha dx + 2v \int_{n/2}^n x^\alpha dx \right) + 2u \\ &= \tilde{K} \left(2u \frac{n^{\alpha+1} - 1}{\alpha + 1} + 2v \frac{n^{\alpha+1} - (n/2)^{\alpha+1}}{\alpha + 1} \right) + 2u \\ &= \tilde{K} n^{\alpha+1} \frac{2u + 2v - v2^{-\alpha}}{\alpha + 1} - \frac{2\tilde{K}u}{\alpha + 1} + 2u \\ &= \tilde{K} n^{\alpha+1} - \frac{2\tilde{K}u}{\alpha + 1} + 2u. \end{aligned}$$

The first inequality is valid by noticing that

$$\max(z_1, z_n) = \max(1, z_n) \leq \tilde{K}(n - 1)^\alpha = \tilde{K} \max(0, (n - 1)^\alpha).$$

The second inequality uses Lemma 4.5. Choose $\tilde{K} = \alpha + 1 > 1$, then $z_{n+2} \leq \tilde{K}n^\alpha \leq \tilde{K}(n + 2)^\alpha$. $\square$

Remark 4.1. In the proof of the lower bound, we use the fact that $(n + L)^{\alpha+1}$ dominates $n(n + L + 2)^{\alpha+1}$ when L is large enough. Similarly, $(n - L)^{\alpha+1} \ll n(n - L + 2)^{\alpha+1}$ for large L . However, applying the same idea to the upper bound would start the induction only after some $n > L$. We avoid this by choosing $L = 1$ and saving a fixed budget $2u$ in the first term, which also gives a useful algebraic simplification.

We now prove the exact asymptotics stated in Theorem 4.3. A natural goal is to remove the maximum operator. If $z_1 = z_2 = 1$, then the sequence would be increasing from the beginning, and induction would allow us to eliminate the maximum operator. However, the initial conditions $z_1 = 1$ and $z_2 = 0$ make the sequence (z_n) fluctuate during the first iterations, and the index at which it becomes

4.2 Asymptotics of the lower bound recurrence

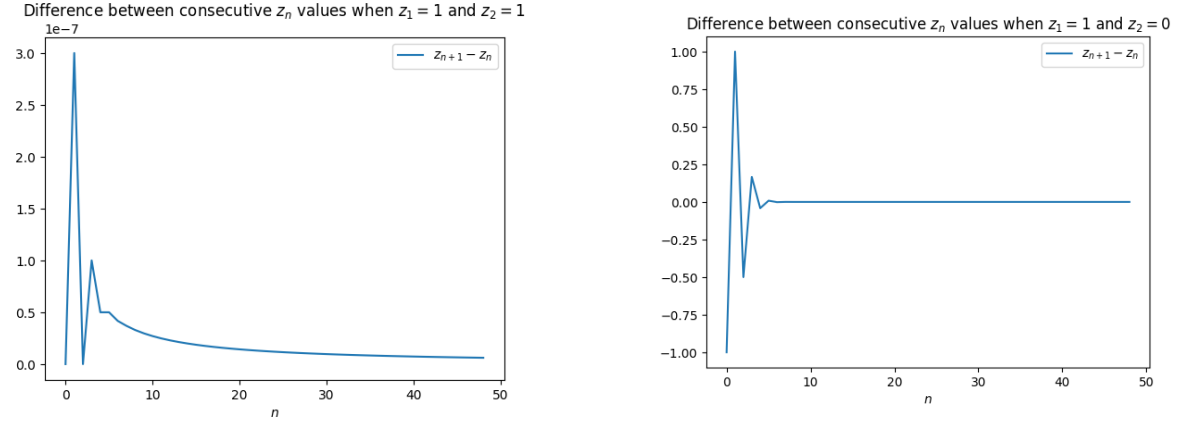


Figure 4.2: First consecutive differences of (z_n) in two initializations ($u = 0.5000001, v = 0.000001$)

increasing depends on the parameters u and v . An extreme case of a long fluctuation period occurs when $u \rightarrow 1/2+$ and $v \rightarrow 0+$. We illustrate this phenomenon and compare the two cases in Figure 4.2.

Figure 4.2 compares the consecutive differences of the same recurrence under two close initializations, $z_1 = z_2 = 1$ and $z_1 = 1, z_2 = 0$, in the difficult regime $u = 0.5000001$ and $v = 0.000001$. The second initialization can create a long initial fluctuation, which explains why monotonicity cannot simply be assumed from the beginning.

Eventual monotonicity is sufficient to remove the maximum operator. Assume that there exists N_1 such that $z_{n+1} \geq z_n$ for all $n \geq N_1$. Since (z_n) is unbounded, there exists $N_2 \geq N_1$ such that for all $n \geq N_2$, we have both $z_{n+1} \geq \max_{1 \leq i \leq N_1} z_i$ and $z_{n+1} \geq z_n$. That is,

$$\forall n \geq N_2, \quad z_{n+1} \geq \max_{1 \leq i \leq n} z_i.$$

Taking $N_3 = 2N_2 - 1$, we have

$$\forall n \geq N_3, \quad \forall i \in \left\{ \left\lceil \frac{n+1}{2} \right\rceil, \dots, n \right\}, \quad z_i \geq z_{n+1-i}.$$

Under the stronger parameter condition (4.7), Theorem 4.11 below proves that such an index exists. The question about the first index where the fluctuations stop is itself interesting, and we leave it open.

Open question 1 *In terms of u and v , what is the first index N such that $(z_n)_{n \geq N}$ is increasing?*

We proceed to proving eventual monotonicity. The idea is to use loose inequalities to bootstrap basic properties, then upgrade step by step with tighter inequalities. The next lemma uses a loose inequality $\max(a, b) \leq a + b$ to show that the difference between consecutive terms vanishes. To begin with, let us define for $n \geq 1$ the difference

$$\Delta z_n = z_n - z_{n-1}.$$

Lemma 4.6 (Vanishing difference) *We have $|\Delta z_n| \rightarrow 0$.*

4.2 Asymptotics of the lower bound recurrence

Proof. For $n \geq 2$, we have

$$\begin{aligned} nz_{n+2} &= 2u \sum_{i=1}^n z_i + v \sum_{i=1}^n \max(z_i, z_{n+1-i}), \\ (n-1)z_{n+1} &= 2u \sum_{i=1}^{n-1} z_i + v \sum_{i=1}^{n-1} \max(z_i, z_{n-i}). \end{aligned}$$

Subtracting the equations side by side, we get

$$\begin{aligned} n\Delta z_{n+2} &= 2uz_n + v \max(z_n, z_1) - z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_i, z_{n+1-i}) - \max(z_i, z_{n-i})) \\ &= 2uz_n + v \max(z_n, z_1) - z_{n+1} + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)). \end{aligned} \tag{4.10}$$

Using $\max(z_n, z_1) \leq z_n + z_1$, we have

$$\begin{aligned} n\Delta z_{n+2} &\leq 2uz_n + v(z_n + z_1) - vz_{n+1} + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)) \\ &\leq 2uz_n + vz_1 + v(z_n - z_{n+1}) + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+1}) - \max(z_{n-i}, z_i)) \end{aligned}$$

Using $|\max(a, b) - \max(a, c)| \leq |b - c|$, we have

$$\begin{aligned} n|\Delta z_{n+2}| &\leq 2uz_n + vz_1 + v|\Delta z_n| + v \sum_{i=1}^{n-1} |\Delta z_i| = 2uz_n + vz_1 + v \sum_{i=1}^n |\Delta z_i| \\ &\leq 2uz_n + vz_1 + nv \sup_{1 \leq i \leq n} |\Delta z_i|. \end{aligned}$$

Since $u < 1$, we have $\alpha < 1$. By $z_n = \Theta(n^\alpha)$, we have $\frac{1}{n}(2uz_n + vz_1) \rightarrow 0$, or

$$|\Delta z_{n+2}| \leq v \sup_{1 \leq i \leq n} |\Delta z_i| + o(1).$$

We have $v < 1$ since otherwise the condition $2u + v > 1$ would be violated. Therefore $(|\Delta z_i|)$ is bounded. Let $M = \sup_{n \geq 1} |\Delta z_n|$. For every $\epsilon > 0$, there exists N such that for all $n \geq N$, $|\Delta z_n| < M + \epsilon$. Therefore,

$$|\Delta z_{n+2}| \leq v(M + \epsilon) + o(1).$$

Letting $n \rightarrow \infty$ gives $M \leq v(M + \epsilon)$. Since $\epsilon > 0$ is arbitrary, we have $M = 0$. $\square$

Since (z_n) is unbounded, we have the following corollary.

Corollary 4.7 (Eventual exceedance) *Let α be the solution to (4.6). Then, for every $M > 0$, there exists N such that for all $n \geq N$, $z_n > M$.*

Lemma 4.8 (Difference identity) *For all sufficiently large n , there exist numbers $\theta_{n,i} \in [0, 1]$ for $1 \leq i \leq n-1$ such that*

$$n\Delta z_{n+2} = (2u + v - 1)z_n - \Delta z_n + v \sum_{i=1}^{n-1} \theta_{n,i} \Delta z_i. \tag{4.11}$$

Proof. We start from (4.10)

$$n\Delta z_{n+2} = 2uz_n + v \max(z_n, z_1) - z_{n+2} + v \sum_{i=1}^{n-1} (\max(z_i, z_{n+2-i}) - \max(z_i, z_{n-i})).$$

4.2 Asymptotics of the lower bound recurrence

By Corollary 4.7, we have $\max(z_n, z_1) = z_n$ for all sufficiently large n . Hence, for all sufficiently large n ,

$$n\Delta z_{n+2} = (2u + v - 1)z_n - \Delta z_n + v \sum_{i=1}^{n-1} (\max(z_{n-i}, z_{i+2}) - \max(z_{n-i}, z_i)).$$

For every $a \geq 0$, the function $x \mapsto \max(a, x)$ is increasing and 1-Lipschitz. Hence for each n, i there is some $\theta_{n,i} \in [0, 1]$ satisfying (4.11). $\square$

At this step, we have isolated the difference sequence (Δz_n) from the original sequence (z_n) . Our next goal is to prove that (Δz_n) is *eventually positive*. This would follow if, in (4.11), a lower bound for $-\Delta z_n + v \sum_{i=1}^{n-1} \theta_{n,i} \Delta z_i$ could be absorbed into $(2u + v - 1)z_n$. We already have $|\Delta z_n| = o(z_n)$ since (Δz_n) is bounded.

Because little is known about $\theta_{n,i}$, we can only use a worst-case bound. A basic absolute-value estimate would require proving that $\sum_{i=1}^n |\Delta z_i| = o(z_n)$. However, if $z_n \sim Cn^\alpha$ and z_n is eventually increasing, then $\sum_{i=1}^n |\Delta z_i| \sim Cn^\alpha$. We therefore need a tighter lower bound. We use the *negative variation* of Δz_n , defined for every $x \in \mathbb{R}$ as

$$x_- = \max(-x, 0). \quad (4.12)$$

We have

$$-x \leq x_- \text{ and } 0 \leq x_-. \quad (4.13)$$

Define the negative variation series of (Δz_n) as

$$V_n := \sum_{i=1}^n (\Delta z_i)_-. \quad (4.14)$$

Then (V_n) is increasing. We state the criterion for eventual increase of (z_n) in terms of (V_n) as follows.

Lemma 4.9 (Negative variation criterion) *If $V_n = o(z_n)$, then (z_n) is eventually increasing.*

Proof. By Lemma 4.8, for all sufficiently large n ,

$$n\Delta z_{n+2} \geq (2u + v - 1)z_n - |\Delta z_n| - vV_n.$$

Since Δz_n is bounded and $z_n \rightarrow \infty$, we have $|\Delta z_n| = o(z_n)$. By assumption, $V_n = o(z_n)$. Hence for all sufficiently large n ,

$$n\Delta z_{n+2} \geq \frac{1}{2}(2u + v - 1)z_n > 0$$

$\square$

Numerical experiments suggest that (V_n) is likely bounded. For the proof of eventual monotonicity, however, the following weaker estimate is enough.

Lemma 4.10 *Assume that $v > 0$. We have $V_n = O(n^v)$.*

Proof. Again, we start from (4.11). There exists N such that for all $n \geq N$,

$$n\Delta z_{n+2} = (2u + v - 1)z_n - \Delta z_n + v \sum_{j=1}^{n-1} \theta_{n,j} \Delta z_j.$$

4.2 Asymptotics of the lower bound recurrence

Let $M = \sup |\Delta z_n|$. If $\Delta z_{n+2} \geq 0$, then $(\Delta z_{n+2})_- = 0$. Otherwise, $(\Delta z_{n+2})_- = -\Delta z_{n+2}$, and therefore

$$\begin{aligned}
n(\Delta z_{n+2})_- &= \Delta z_n - (2u + v - 1)z_n - v \sum_{j=1}^{n-1} \theta_{n,j} \Delta z_j \\
&\leq |\Delta z_n| - (2u + v - 1)z_n + v \sum_{j=1}^{n-1} \theta_{n,j} (\Delta z_j)_- \quad (-\Delta z_j \leq (\Delta z_j)_-, \forall j) \\
&\leq |\Delta z_n| + v \sum_{j=1}^{n-1} (\Delta z_j)_- \quad (0 \leq \theta_{n,j} \leq 1) \\
&\leq M + vV_{n-1}.
\end{aligned}$$

Thus, in either case,

$$(\Delta z_{n+2})_- \leq \frac{v}{n} V_{n-1} + \frac{M}{n}.$$

Since (V_n) is increasing, we get

$$V_{n+2} = V_{n+1} + (\Delta z_{n+2})_- \leq \left(1 + \frac{v}{n}\right) V_{n+1} + \frac{M}{n}. \quad (4.15)$$

Reindexing, for every $m \geq N + 2$,

$$V_m + \frac{M}{v} \leq \left(1 + \frac{v}{m-2}\right) \left(V_{m-1} + \frac{M}{v}\right).$$

Iterating this inequality gives

$$V_m + \frac{M}{v} \leq \prod_{i=N}^{m-2} \left(1 + \frac{v}{i}\right) \left(V_{N+1} + \frac{M}{v}\right).$$

It remains to estimate the product. Taking the logarithm gives

$$\begin{aligned}
\ln \left(\prod_{i=N}^{m-2} \left(1 + \frac{v}{i}\right) \right) &= \sum_{i=N}^{m-2} \ln \left(1 + \frac{v}{i}\right) \\
&\leq \sum_{i=N}^{m-2} \frac{v}{i} = v(H_{m-2} - H_{N-1}) \\
&\leq vH_m \leq v \ln m + C,
\end{aligned}$$

where $H_n = \sum_{i=1}^n \frac{1}{i} = \ln n + \gamma + O(1/n)$ is the n -th harmonic number and C is some constant. Exponentiating both sides yields

$$\prod_{i=N}^{m-2} \left(1 + \frac{v}{i}\right) \leq e^C m^v = O(m^v).$$

□

We state our checkpoint as a theorem.

Theorem 4.11 *If $0 < v < \alpha$, then (z_n) is eventually increasing.*

Proof. By the previous lemma, $V_n = O(n^v) = o(n^\alpha) = o(z_n)$, since $v < \alpha$ and $z_n = \Theta(n^\alpha)$. The result follows from the negative variation criterion. □

4.2 Asymptotics of the lower bound recurrence

When $v = 0$, the recurrence (4.4) reduces to the full-history recurrence (4.3), whose asymptotics are handled by Proposition A.1. Thus the negative-variation argument is only needed for $v > 0$.

We now express the condition $0 < v < \alpha$ in terms of u and v . Define

$$f(\alpha) = \alpha + 1 - 2(u + v) + v2^{-\alpha}, \alpha \in [0, 1].$$

Then $f'(\alpha) = 1 - v2^{-\alpha} \ln 2 > 1 - \ln 2 > 0$ for all $\alpha > 0$. Hence $v < \alpha$ is equivalent to $f(v) < 0$ i.e.,

$$v + 1 - 2(u + v) + v2^{-v} < 0 \iff u > \frac{1}{2}(1 - v + v2^{-v}).$$

For $v > 0$, if we impose conditions (4.7), then the above inequality is satisfied. The case $v \geq \alpha$ with $v > 0$ is left open. Once (z_n) is eventually increasing, it satisfies a simpler recurrence without the whole history.

Lemma 4.12 *There exists $N \geq 1$ such that the sequence $(z_n)_{n \geq N}$ defined by 4.4 satisfies the recurrence*

$$nz_{n+2} - (n-1)z_{n+1} = 2(u+v)z_n - vz_{\lfloor n/2 \rfloor}. \quad (4.16)$$

with $z_{N+1} \geq z_N \geq \dots \geq z_{\lfloor n/2 \rfloor} \geq 1$.

Proof. Since (z_n) is eventually increasing, there exists N such that for all $n \geq N$, we have $z_n \geq z_i$ for all $1 \leq i \leq n-1$ and $z_n \geq 1$. Hence, for all $n \geq N+1$,

$$\begin{aligned} nz_{n+2} &= 2u \sum_{i=1}^n z_i + 2v \sum_{i=\lfloor n/2 \rfloor + 1}^n z_i + \delta_n v z_{\lfloor n/2 \rfloor} \\ (n-1)z_{n+1} &= 2u \sum_{i=1}^{n-1} z_i + 2v \sum_{i=\lfloor (n-1)/2 \rfloor + 1}^{n-1} z_i + \delta_{n-1} v z_{\lfloor (n-1)/2 \rfloor}, \end{aligned}$$

where $\delta_n = 1$ if n is odd and 0 if n is even. If $n = 2k$, then $\lfloor n/2 \rfloor = \lfloor (n-1)/2 \rfloor = k$, $\delta_n = 0$ and $\delta_{n-1} = 1$. Hence,

$$2kz_{2k+2} - (2k-1)z_{2k+1} = 2(u+v)z_{2k} - vz_k.$$

If $n = 2k+1$, then $\lfloor n/2 \rfloor = \lfloor (n-1)/2 \rfloor = k$, $\delta_n = 1$ and $\delta_{n-1} = 0$. Hence,

$$(2k+1)z_{2k+3} - (2k)z_{2k+2} = 2(u+v)z_{2k+1} - 2vz_{k+1} + vz_{k+1} = (2k+1)(u+v)z_{2k+1} - vz_{k+1}.$$

Collecting the two cases and noticing that $\left\lceil \frac{n-1}{2} \right\rceil = \left\lfloor \frac{n}{2} \right\rfloor$, we arrive at (4.16). $\square$

With the recurrence (4.16), we can now prove the exact asymptotics of (z_n) . We will use a dyadic argument to show that the normalized difference sequence (Δx_n) is absolutely summable by analyzing it on the intervals $[2^m, 2^{m+1})$.

Proof of Theorem 4.3. Let $x_n = \frac{z_n}{(n-2)^\alpha}$ for $n \geq N$. We will prove that (x_n) converges by proving that (Δx_n) is absolutely summable, i.e., $\sum_{n \geq N} |\Delta x_n|$ converges. Note that $x_n = \Theta(1)$ since $z_n = \Theta(n^\alpha)$. For $n \geq N \geq 6$ (so that $x_{\lfloor n/2 \rfloor}$ is defined), we have

$$\begin{aligned} n^{\alpha+1}x_{n+2} - (n-1)^{\alpha+1}x_{n+1} &= 2(u+v)(n-2)^\alpha x_n - v \left(\left\lfloor \frac{n}{2} \right\rfloor - 2 \right)^\alpha x_{\lfloor n/2 \rfloor} \\ \iff x_{n+2} &= \left(1 - \frac{1}{n} \right)^{\alpha+1} x_{n+1} + \frac{2(u+v)}{n} \left(1 - \frac{2}{n} \right)^\alpha x_{n-1} - \frac{v}{n} \left(\frac{\lfloor n/2 \rfloor - 2}{n} \right)^\alpha x_{\lfloor n/2 \rfloor}. \end{aligned}$$

4.2 Asymptotics of the lower bound recurrence

We have

$$\begin{aligned} \left(1 - \frac{1}{n}\right)^{\alpha+1} &= 1 - \frac{\alpha+1}{n} + O\left(\frac{1}{n^2}\right), \\ \frac{1}{n} \left(1 - \frac{2}{n}\right)^\alpha &= \frac{1}{n} + O\left(\frac{1}{n^2}\right), \\ \frac{1}{n} \left(\frac{\lfloor n/2 \rfloor - 2}{n}\right)^\alpha &= \frac{2^{-\alpha}}{n} + O\left(\frac{1}{n^2}\right). \end{aligned}$$

The last expansion follows from $\frac{1}{2} - \frac{3}{n} \leq \frac{\lfloor n/2 \rfloor - 2}{n} \leq \frac{1}{2} - \frac{1}{n}$. Since α solves (4.6), we have

$$\begin{aligned} x_{n+2} - x_{n+1} &= -\frac{\alpha+1}{n}x_{n+1} + \frac{2(u+v)}{n}x_n - \frac{v2^{-\alpha}}{n}x_{\lfloor n/2 \rfloor} + O\left(\frac{1}{n^2}\right) \\ &= -\frac{2(u+v)}{n}(x_{n+1} - x_n) + \frac{v2^{-\alpha}}{n}(x_n - x_{\lfloor n/2 \rfloor}) + O\left(\frac{1}{n^2}\right). \end{aligned}$$

There exists a constant $K_1 > 0$ such that for every $n \geq N' - 2 \geq N$,

$$|\Delta x_{n+2}| \leq \frac{2(u+v)}{n}|\Delta x_{n+1}| + \frac{v2^{-\alpha}}{n} \sum_{i=\lfloor n/2 \rfloor+1}^n |\Delta x_i| + \frac{K_1}{(n+2)^2}.$$

Equivalently, for $n \geq N'$,

$$|\Delta x_n| \leq \frac{2(u+v)}{n-2}|\Delta x_{n-1}| + \frac{v2^{-\alpha}}{n-2} \sum_{i=\lfloor n/2 \rfloor}^{n-2} |\Delta x_i| + \frac{K_1}{n^2}.$$

Let $u_m = \max_{2^m \leq n < 2^{m+1}} |\Delta x_n|$. Then, for every $n \in [2^m, 2^{m+1})$, we have

$$\begin{aligned} |\Delta x_{n-1}| &\leq \max_{2^{m-1} \leq i < 2^m} |\Delta x_i| \leq \max(u_{m-1}, u_m), \\ \frac{1}{n-2} \sum_{i=\lfloor n/2 \rfloor}^{n-2} |\Delta x_i| &\leq \frac{1}{2} \max_{2^{m-1} \leq i < 2^m} |\Delta x_i| \leq \frac{1}{2} \max(u_{m-1}, u_m). \end{aligned}$$

Therefore,

$$|\Delta x_n| \leq \left(\frac{2(u+v)}{n-2} + v2^{-\alpha-1} \right) \max(u_{m-1}, u_m) + \frac{K_1}{2^{2m}}.$$

Taking the maximum over $n \in [2^m, 2^{m+1})$, we have

$$u_m \leq \left(\frac{2(u+v)}{n-2} + v2^{-\alpha-1} \right) \max(u_{m-1}, u_m) + \frac{K_1}{2^{2m}}.$$

Since $v2^{-\alpha} = 2(u+v) - \alpha - 1 < 1$, we can choose m_0 large enough such that $2^{m_0} \geq N'$, and there exists γ_1 such that

$$\frac{2(u+v)}{n-2} + v2^{-\alpha-1} < \gamma_1 < \frac{1}{2}$$

for all $m \geq m_0$ and $n \in [2^m, 2^{m+1})$. If $u_{m-1} > u_m$, then $u_m \leq \gamma_1 u_{m-1} + \frac{K_1}{2^{2m}}$. If $u_{m-1} \leq u_m$, then

4.3 Asymptotics of the stochastic recurrence

$u_m \leq \frac{K_1}{2^{2m}(1-\gamma_1)}$. In both cases, we have

$$\begin{aligned} u_m &\leq \gamma_1 u_{m-1} + \frac{K_1}{2^{2m}(1-\gamma_1)} \\ &\leq \sum_{i=m_0+1}^m \gamma_1^{m-i} \frac{K_1}{2^{2i}(1-\gamma_1)} + \gamma_1^{m-m_0} u_{m_0} \\ &< \frac{K_1}{(1-\gamma_1)} \sum_{i=m_0+1}^m \gamma_1^{m-i} \left(\frac{1}{4}\right)^i + \gamma_1^{m-m_0} u_{m_0}. \end{aligned}$$

Choose γ_2 such that $\max\left(\gamma_1, \frac{1}{4}\right) < \gamma_2 < \frac{1}{2}$. We have

$$u_m < \frac{K_1}{(1-\gamma_1)} \sum_{i=m_0+1}^m \gamma_2^m + \gamma_2^{m-m_0} u_{m_0} \leq K_2 m \gamma_2^m,$$

for some constant $K_2 > 0$. Choose γ_3 such that $\gamma_2 < \gamma_3 < \frac{1}{2}$. Then there exists $m_1 > m_0$ such that for every $m \geq m_1$, we have $K_2 m \gamma_2^m < K_2 \gamma_3^m$, implying that

$$u_m < K_2 \gamma_3^m.$$

Therefore,

$$\sum_{n=N'+1}^{\infty} |\Delta x_m| \leq \sum_{m=\lfloor \log_2(N') \rfloor}^{\infty} \sum_{n=2^m}^{2^{m+1}-1} |\Delta x_n| < \sum_{m=\lfloor \log_2(N') \rfloor}^{m_1-1} 2^m u_m + K_2 \sum_{m=m_1}^{\infty} (2\gamma_3)^m < \infty.$$

Hence, (x_n) converges to some constant $C > 0$. Thus, $z_n \sim Cn^\alpha$. $\square$

4.3 Asymptotics of the stochastic recurrence

4.3.1 Proof strategy

The stochastic recurrence (3.5) resembles its deterministic counterpart (4.4). Since (4.4) grows as a pure power of n , it is natural to conjecture the same behavior for the expectation in (4.1). Numerical experiments based on Monte Carlo simulation support this conjecture, as shown in Figure 4.3. For each case of (u, v) , we compute the exponent α using the algorithm expressed in (4.20), and the constant $C = z_n/n^\alpha$, where n is the number of iterations. More details of the algorithm are given later in this section.

The growth exponent of y_n is strictly larger than the solution to (4.6). This suggests that, in the long term, $\mathbb{E}[\max(Y_i, Y_j)]$ remains more selective than $\max(\mathbb{E}[Y_i], \mathbb{E}[Y_j])$, especially when i and j are close to each other.

In this section, we present an ongoing attempt to prove the weakest asymptotic result $y_n = n^{\alpha^*+o(1)}$. We adapt the contraction method (e.g., [12]). The original idea is as follows.

1. Guess the scaling factor and derive the normalized recurrence.
2. Derive a limiting fixed-point equation from the normalized recurrence.
3. Prove that the limiting equation has a unique solution in a suitable metric space using a contraction argument.

4.3 Asymptotics of the stochastic recurrence

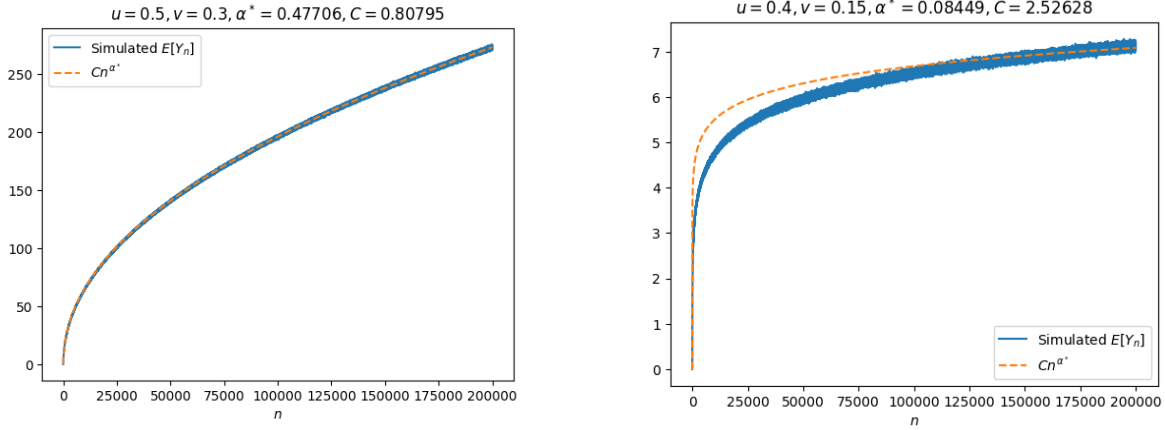


Figure 4.3: The growth of y_n compared with a power Cn^α

4. Prove that the normalized sequence converges in distribution to that unique solution.

We first explain why the contraction method does not apply directly. Let $X_n = Y_n/n^\alpha$ for $\alpha > 0$. The recurrence for X_n and the limiting equation are as follows.

$$X_{n+2} \stackrel{d}{=} I \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)} + \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) + J \max \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)}, \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \quad (4.17)$$

Lemma 4.13 (Limiting equation) *Suppose that $X_n \xrightarrow{d} X$. Then, X satisfies the following distributional equation*

$$X \stackrel{d}{=} I \left(U^\alpha X^{(1)} + (1-U)^\alpha X^{(2)} \right) + J \max(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)}), \quad (4.18)$$

where $U \sim \text{Unif}[0, 1]$.

The proof of Lemma 4.13 is given in Appendix A.2.1. Let $\mathcal{D}$ be the space of probability measures on $\mathbb{R}_+$. Our first goal is to prove that there exists a unique solution (X^*, α^*) to (4.18) in some subspace of $\mathcal{D}$. The fixed-point equation (4.18) always has a degenerate solution δ_0 . The question is therefore to choose a suitable subspace that excludes δ_0 . Define the operator $T_\alpha : \mathcal{D} \rightarrow \mathcal{D}$ induced by the right-hand side of (4.18) as

$$T_\alpha(\mu) = \mathcal{L} \left(I \left(U^\alpha X^{(1)} + (1-U)^\alpha X^{(2)} \right) + J \max(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)}) \right), \quad X^{(1)}, X^{(2)} \stackrel{\text{iid}}{\sim} \mu. \quad (4.19)$$

If T_α is a contraction with respect to the Wasserstein metric for some α , then the Banach fixed-point theorem guarantees that $T_\alpha^n(\mu_1) \xrightarrow{d} 0$. While the Zolotarev metric [20] allows us to choose a subspace excluding δ_0 , it requires certain first moments to remain fixed—a condition T_α violates, as it does not preserve the mean. The original contraction method requires choosing a more suitable metric space, which we consider intractable.

The evolution of an initial distribution under T_α can be understood as follows. At each iteration, T_α changes the shape of the distribution μ_n . Depending on the “dispersion” of μ_n , T_α either increases or decreases its mean. If α is too large, T_α drives the mean to 0 before the distribution becomes sufficiently dispersed. Conversely, if α is too small, T_α pushes the mean to infinity once the distribution is sufficiently dispersed.

4.3 Asymptotics of the stochastic recurrence

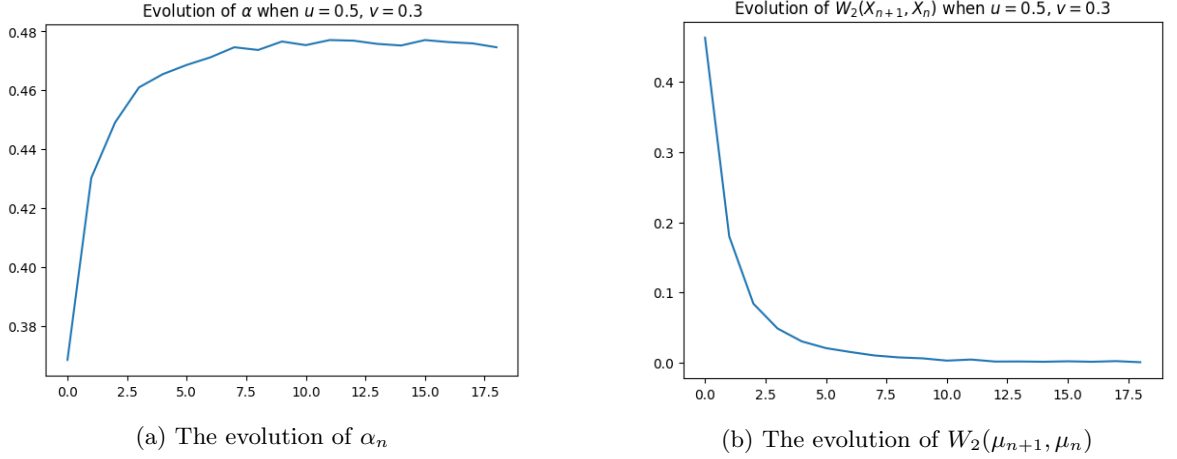


Figure 4.4: Results of the mean-preserving recurrence (4.20) with $u = 0.5$ and $v = 0.3$

The mean is not positively correlated with the need for a larger α . Let $\mu = \delta_2$ and $\nu = 2\text{Ber}(1/2)$, so that $\mathbb{E}_{X \sim \mu}[X] = 2 > 1 = \mathbb{E}_{Y \sim \nu}[Y]$. If α solves (4.6), then $\mathbb{E}_{X \sim \mu}[T_\alpha(X)] = \mathbb{E}_{X \sim \mu}[X]$, but $\mathbb{E}_{Y \sim \nu}[T_\alpha(Y)] > \mathbb{E}_{Y \sim \nu}[Y]$. Thus, although μ has a larger mean than ν , it requires a smaller α to keep its mean fixed. This is because ν is more dispersed than μ .

The second moment is still not sufficient. One can build a similar counterexample by choosing μ and ν with the same mean, $\mathbb{E}_{X \sim \mu}[X^2] > \mathbb{E}_{Y \sim \nu}[Y^2]$, and $\mathbb{E}_{X \sim \mu}[X^3] < \mathbb{E}_{Y \sim \nu}[Y^3]$. If the third moments differ enough compared with the second moments, then $\mathbb{E}_{X \sim \mu}[T_\alpha(X)] < \mathbb{E}_{Y \sim \nu}[T_\alpha(Y)]$. Thus no finite number of moments is sufficient to determine the correct α that keeps the mean fixed.

For each distribution μ of finite mean, there exists a unique α that keeps the mean fixed. Since T_α is homogeneous, we consider the space $\mathcal{D}(1)$ of probability measures on $\mathbb{R}_+$ with mean 1. We state the property below and leave the proof to Appendix A.2.2.

Proposition 4.14 *Define the function $f : \mathcal{D}(1) \times (0, 1] \rightarrow \mathbb{R}_+$ as*

$$f(\mu, \alpha) = \mathbb{E}_{Y \sim T_\alpha(\mu)}[X] = \frac{2u}{\alpha + 1} + v \mathbb{E}_{X^{(1)}, X^{(2)} \sim \mu} \left[\max \left(U^\alpha X^{(1)}, (1 - U)^\alpha X^{(2)} \right) \right].$$

For every $\mu \in \mathcal{D}_2(1)$, there exists a unique $\alpha \in [\alpha_0, 2(u + v) - 1]$, where α_0 solves (4.6), such that $f(\mu, \alpha) = 1$. Moreover, the range is sharp:

$$\forall \alpha \in (0, \alpha_0), f(\mu, \alpha) > 1 \text{ and } \forall \epsilon \in (0, 2u + v - 1 - \alpha_0), \exists \mu \in \mathcal{D}_2(1), f(\mu, \alpha) = 1.$$

A promising approach for finding the true α is to let the distribution and its mean-preserving operator evolve together. By Proposition 4.14, the function $\alpha : \mathcal{D}(1) \rightarrow (0, 1]$ such that $\mathbb{E}_{X \sim \mu}[T_{\alpha(\mu)}(X)] = 1$ is well-defined. We can then define the mean-preserving recurrence as follows.

$$\mu_{n+1} = T_{\alpha_n}(\mu_n), \quad \alpha_n = \alpha(\mu_n) \quad \mu_1 = \delta_1, \quad \alpha_1 = \alpha(\mu_1). \quad (4.20)$$

The evolution of α and the 2-Wasserstein distance between consecutive distributions following the recurrence is shown in Figure 4.4. Fluctuations in the final iterations are likely a side effect of the Monte Carlo simulation, since they become more stable when the number of samples is increased. However, proving that (μ_n, α_n) converges may be challenging. We intended to track a quantity that evolves monotonically with the iterations. Experiments suggest in particular that the higher moments increase through iterations, but we could not complete this approach. Our alternative strategy takes advantage of Tychonoff's theorem, a non-constructive topological existence result. We summarize our proof strategy as follows, and proceed with the first step in the next subsection.

4.3 Asymptotics of the stochastic recurrence

1. Prove that there exists a unique solution (X^*, α^*) to (4.18).
2. Prove that for every $\epsilon > 0$, $\mathbb{E} [Y_n/n^{\alpha^*+\epsilon}] \rightarrow 0$ and $\mathbb{E} [Y_n/n^{\alpha^*-\epsilon}] \rightarrow \infty$.
3. Prove that (4.20) converges to (X^*, α^*) .

In the next subsection, we present a partial result towards the first step.

4.3.2 A fixed-point approach: existence

Since the contraction argument may not be applicable to proving uniqueness of the fixed point, we decompose the problem into proving existence and then uniqueness. Existence is proved using Tychonoff's fixed-point theorem.

Theorem 4.15 (Tychonoff [19, Exercise 2.5c]) *Let $\mathcal{C}$ be a nonempty, compact, convex subset of a locally convex space $\mathcal{M}$. Let $f : \mathcal{C} \rightarrow \mathcal{C}$ be a continuous mapping. Then f has a fixed point.*

We now prepare the ingredients for the Tychonoff fixed-point theorem. Ideally, the subset $\mathcal{C}$ should be as small as possible while still containing (μ_n) in (4.20) and $(\mathcal{L}(X_n))$ for X_n defined in (4.17) for all $\alpha \in (2u + v - 1, 2u + v)$. This makes uniqueness more plausible once existence is established. Define $\Phi : \mathcal{D}(1) \rightarrow \mathcal{D}(1)$ by

$$\Phi(\mu) = T_{\alpha(\mu)}(\mu), \quad \forall \mu \in \mathcal{D}(1). \quad (4.21)$$

In the following, we will show that the subset $\mathcal{C}$ can be restricted to the set of distributions with uniformly bounded second moment. For $M > 0$, define

$$\mathcal{D}_2(1, M) = \left\{ \mu \in \mathcal{D}(1) : \int_0^\infty x^2 d\mu \leq M \right\}. \quad (4.22)$$

Proposition 4.16 *Assume that $u > 1/2$. Then there exists $M_0 > 0$ such that for every $M \geq M_0$, the operator Φ defined in (4.21) maps $\mathcal{D}_2(1, M)$ into itself.*

The proof is given in Appendix A.2.3. Again, the condition $u > 1/2$ is crucial. Choose $M = \max(1, M_0)$. In $\mathcal{D}_2(1, M)$, the range in Proposition 4.14 can be slightly improved to $\left[\alpha_0, 2(u + v) - 1 - \frac{v}{2M} \right)$.

The next ingredient is the host vector space $\mathcal{M}$. To avoid excessive elaboration, we combine two standard points of view. From probability theory, weak convergence can be defined using the space $C_b(\mathbb{R})$ of bounded, continuous test functions [2, Chap. 1]. From functional analysis, local convexity is obtained by equipping $C_b(\mathbb{R})$ with a topology and defining $\mathcal{M}$ as its dual space of continuous linear functionals. In this way, the weak topology on $\mathcal{M}$ (called the weak*-topology in functional analysis) is induced by a family of seminorms $\{p_f\}_{f \in C_b(\mathbb{R})}$, which makes $\mathcal{M}$ a locally convex space. Finally, it is enough to have $\mathcal{D}_2(1, M)$ compact in $\mathcal{M}$. However, just as we want $\mathcal{D}_2(1, M)$ to be only large enough to contain our distributions of interest, we also want $\mathcal{M}$ to be only large enough to contain $\mathcal{D}_2(1, M)$, since a smaller host space is usually better behaved. We summarize the above discussion as follows, leaving the details to Appendix A.1.2 and Appendix A.2.5.

Theorem 4.17 (Dual space of $C_b(\mathbb{R})$ with the strict topology [3]) *Equip $C_b(\mathbb{R})$ with the strict topology. Then the dual space is isometrically isomorphic to the space $\mathcal{M}(\mathbb{R})$ of bounded Radon measures on $\mathbb{R}$. Moreover, $\mathcal{M}(\mathbb{R})$ is locally convex with respect to the weak topology.*

Proposition 4.18 *The set $\mathcal{D}_2(1, M)$ is a nonempty, compact, and convex subset of the space $\mathcal{M}(\mathbb{R})$ defined in Theorem 4.17.*

4.3 Asymptotics of the stochastic recurrence

Proposition 4.19 *The operator Φ defined in (4.21) is continuous on $\mathcal{D}_2(1, M)$ with respect to the weak topology.*

A useful property of $\mathcal{D}_2(1, M)$ is that for any $\mu \in \mathcal{D}_2(1, M)$ and any $R > 0$, by Chebyshev's inequality, we have

$$\int_R^\infty x \, d\mu \leq \int_R^\infty x \left(\frac{x}{R}\right) d\mu = \frac{1}{R} \int_R^\infty x^2 \, d\mu \leq \frac{1}{R} \int_0^\infty x^2 \, d\mu \leq \frac{M}{R}.$$

This implies tightness and uniform integrability of $\mathcal{D}_2(1, M)$. Tightness is used in proving compactness, and uniform integrability is used in proving continuity of Φ by identifying weak continuity with W_1 -continuity.

These ingredients allow us to apply Tychonoff's fixed-point theorem.

Theorem 4.20 *Assume that $u > 1/2$. Then there exists a solution*

$$(\mu^*, \alpha^*) \in \mathcal{D}_2(1, M) \times \left[\alpha_0, 2(u + v) - 1 - \frac{v}{2M} \right),$$

where α_0 solves (4.6), to the fixed-point equation (4.18).

5

Conclusion

This report started from a practical observation: the efficiency of modern algorithms cannot be fully explained by classical textbook techniques alone. In particular, high-performance regular expression matching is no longer explained only by classical automata theory. Engines such as Hyperscan combine automata constructions, graph decompositions, SIMD string matchers, hashing, and verification heuristics. This makes them interesting from two complementary points of view. On the systems side, their performance depends on hardware-aware design choices. On the mathematical side, several of those choices lead to natural statistics of random regular expressions and to recursive distributional equations.

We first reviewed the concepts needed to make this connection precise: regular expressions and finite automata, the Glushkov construction, BST-like distributions on syntax trees, SIMD parallelism, analytic combinatorics, and Monte Carlo simulation. We then traced the main compilation and execution paths of Hyperscan. In the compilation phase, regular expressions are converted to Glushkov NFAs, decomposed by Violet, and organized by Rose into a graph of literals and sub-FAs. In the execution phase, literal matchers act as filters that trigger the remaining automata. This viewpoint suggests that the extracted literals are not merely implementation details: their lengths and interactions directly influence how much work is delegated to SIMD matchers and how much work remains for verification.

The FDR string matcher provided a concrete example of this phenomenon. Its simplified model can be understood as a multi-pattern extension of Shift-Or, where patterns are grouped into buckets, represented through masks, and scanned with parameters such as super-character size and stride. The experiments around this model indicate that an analysis based only on SIMD operations is incomplete. False positives, hashing, and exact verification also have to be accounted for. This gives a first family of future problems: build a faithful probabilistic model of FDR that includes bucket assignment, super-characters, stride, the hash table, and the induced verification cost. Such a model should be compared against measurements on real pattern sets and, when possible, against the current Vectorscan implementation.

The main mathematical part of the report focused on the length of the longest literal extracted from a random regular expression. Under the BST-like model, concatenation leads to sums, alternation leads to maxima, and Kleene star resets the contribution to zero. This gives the stochastic recurrence (3.5). For its expectation (y_n) , we introduced deterministic lower and upper bounds, then studied in detail the tightened lower-bound recurrence (z_n) . The results show that (z_n) grows as a power of n : first in the weaker form $z_n = n^{\alpha+o(1)}$, then as $z_n = \Theta(n^\alpha)$, and finally, under the stronger condition $u > 1/2$, as $z_n \sim Cn^\alpha$, where α is determined by a transcendental equation involving the parameters u and v .

The proof of the exact asymptotics is interesting because the maximum operator cannot be removed

immediately. The initial values create possible early fluctuations, so the proof first establishes eventual monotonicity through a negative-variation argument. This leaves a useful open question even for the deterministic recurrence: determine the first index from which monotonicity holds, and remove or weaken the current parameter restrictions. A long-term goal is to document a repository of techniques applicable to recurrences with sums and maxima, as they appear in many other contexts.

The original stochastic recurrence appears to have a richer asymptotic behavior than the deterministic lower bound. Simulations suggest a pure power growth, but with an exponent larger than the one obtained from the tightened lower bound. The fixed-point approach developed in the last part of the report gives a possible route forward. After normalizing Y_n by n^α , any limiting distribution should satisfy the distributional equation (4.18). A direct contraction argument is obstructed by the degenerate fixed point δ_0 and by the fact that T_α does not preserve the mean. The mean-preserving recurrence (4.20) addresses this by evolving both the distribution and the exponent. We proved an existence result for a fixed point under a bounded second-moment condition, using Tychonoff's fixed-point theorem.

Several directions remain open. The most important one is to complete the asymptotic analysis of the stochastic recurrence: prove that $y_n = n^{\alpha^* + o(1)}$, identify or characterize α^* , and prove convergence of the mean-preserving recurrence.

Overall, the internship shows that Hyperscan is a productive source of theoretical questions. Its optimizations are practical, but many of them expose clean mathematical objects: random syntax trees, sums and maxima in recurrences, dependency graphs, and filtering/verification trade-offs. A complete average-case analysis of such an engine is still out of reach, but the longest-literal problem gives a tractable first step and a concrete path toward more realistic models of modern regular expression matching.



Appendix

A.1 Supplementary results

A.1.1 Asymptotics of recurrences that include a sum of history

Proposition A.1 *Consider the sequence (z_n) defined by the recurrence*

$$z_{n+2} = \frac{K}{n} \sum_{i=1}^n z_i, \quad n \geq 1, \quad (1.1)$$

where $K > 0$. The initial values z_1 and z_2 are nonnegative and not both zero. Then $z_n \sim Cn^{K-1}$ for some constant $C > 0$.

Proof. The recurrence is equivalent to

$$Kz_{n+2} = \sum_{i=1}^n z_i.$$

Let $Z(x) = \sum_{n=1}^{\infty} z_n x^n$ be the generating function of (z_n) . Summing the rewritten recurrence over $n \geq 1$ gives

$$\sum_{n=1}^{\infty} n z_{n+2} x^n = K \sum_{n=1}^{\infty} \sum_{n=1}^{\infty} z_n x^n.$$

For the left-hand side, we have

$$\begin{aligned} \sum_{n=1}^{\infty} n z_{n+2} x^n &= \frac{1}{x} \sum_{n=3}^{\infty} (n-2) z_n x^{n-1} = \frac{1}{x} \left(\sum_{k=3}^{\infty} k z_k x^{k-1} - \frac{2}{x} \sum_{k=3}^{\infty} z_k x^k \right) \\ &= \frac{1}{x} \left(Z'(x) - z_1 - 2z_2 x - \frac{2}{x} (Z(x) - z_1 x - z_2 x^2) \right) \\ &= \frac{Z'(x)}{x} - \frac{2Z(x)}{x^2} + \frac{z_1}{x}. \end{aligned}$$

For the right-hand side, we have

$$K \sum_{n=1}^{\infty} \sum_{n=1}^{\infty} z_n x^n = K \frac{Z(x)}{1-x}.$$

A.1 Supplementary results

Equating the two sides and rearranging, we obtain a first-order linear differential equation

$$Z'(x) - \left(\frac{2}{x} + \frac{Kx}{1-x} \right) Z(x) = -z_1.$$

Multiplying both sides by the integrating factor $I(x) = x^{-2}(1-x)^K e^{Kx}$ gives

$$[I(x)Z(x)]' = -z_1 x^{-2}(1-x)^K e^{Kx} = -z_1 x^{-2} - z_1 \frac{(1-x)^K e^{Kx} - 1}{x^2}.$$

To integrate this from 0 to x without introducing an arbitrary integration constant, we group the singular x^{-2} terms. The local expansion of $Z(x)$ at $x = 0$ is $z_1 x + z_2 x^2 + \mathcal{O}(x^3)$, and the Taylor series of $(1-x)^K e^{Kx}$ begins with $1 - \frac{K}{2}x^2 + \mathcal{O}(x^3)$. Thus, we have the limit

$$\lim_{x \rightarrow 0} \left(I(x)Z(x) - \frac{z_1}{x} \right) = \lim_{x \rightarrow 0} \left(x^{-2}(1-x)^K e^{Kx} Z(x) - \frac{z_1}{x} \right) = z_2.$$

Furthermore, $\lim_{x \rightarrow 0} \frac{(1-x)^K e^{Kx} - 1}{x^2} = -\frac{K}{2}$, confirming that the right-hand integrand is continuous. Moving $-z_1 x^{-2}$ to the left side and integrating from 0 to x via the Fundamental Theorem of Calculus yields

$$I(x)Z(x) - \frac{z_1}{x} - z_2 = -z_1 \int_0^x \frac{(1-t)^K e^{Kt} - 1}{t^2} dt.$$

Multiplying by $I(x)^{-1}$ isolates the closed-form generating function

$$Z(x) = (1-x)^{-K} e^{-Kx} \left(z_2 x^2 + z_1 x - z_1 x^2 \int_0^x \frac{(1-t)^K e^{Kt} - 1}{t^2} dt \right).$$

To extract the asymptotic behavior of (z_n) , we analyze $Z(x)$ near its dominant singularity at $x = 1$. The integral converges on $(0, 1)$, giving the local expansion

$$Z(x) \underset{x \rightarrow 1}{\sim} A e^{-K} (1-x)^{-K},$$

where the leading constant is

$$A = z_2 + z_1 \left(1 - \int_0^1 \frac{(1-t)^K e^{Kt} - 1}{t^2} dt \right).$$

We must ensure singularity cancellation does not occur. For any $t \in (0, 1)$, the inequality $\ln(1-t) < -t$ implies $K \ln(1-t) + Kt < 0$. Exponentiating yields $(1-t)^K e^{Kt} < 1$, so the integrand is strictly negative. Consequently, the integral evaluates to a strictly negative value, making $1 - \int_0^1 \frac{(1-t)^K e^{Kt} - 1}{t^2} dt > 1$. Because $z_1, z_2 \geq 0$ and are not both zero, it follows that $A \geq z_2 + z_1 > 0$. Since $K > 0$, we apply Theorem 2.1. Mapping the $(1-x)^{-K}$ singularity to the coefficients $z_n = [x^n]Z(x)$ yields

$$z_n \sim \frac{A e^{-K}}{\Gamma(K)} n^{K-1}.$$

Defining $C = \frac{A e^{-K}}{\Gamma(K)}$, we have $C > 0$, completing the proof. $\square$

A.1.2 Spaces of probability measures

Definition A.1 (Tightness). A sequence of probability measures (μ_n) on $\mathbb{R}$ is tight if for every $\varepsilon > 0$, there exists $M > 0$ such that $\sup_n \mu_n(\{x : |x| > M\}) < \varepsilon$.

A sequence of random variables (Z_n) is tight if for every $\varepsilon > 0$, there exists $M > 0$ such that $\sup_n \mathbb{P}(|Z_n| > M) < \varepsilon$. $\blacksquare$

Theorem A.2 (Prokhorov's theorem) A sequence of probability measures (μ_n) on $\mathbb{R}$ is tight if and only if it is relatively compact in the weak topology (i.e., its closure is compact).

A.2 Remaining proofs

A.2.1 Lemma 4.13

We need the following lemma.

Lemma A.3 *Let U_n be a uniform random variable on $\{1, 2, \dots, n\}$ and let U be a uniform random variable on $[0, 1]$. Then $\frac{U_n}{n} \xrightarrow{d} U$.*

Proof. Recall the CDF of U

$$F_U(x) = \begin{cases} 0 & x < 0, \\ x & 0 \leq x \leq 1, \\ 1 & x > 1. \end{cases}$$

Let $V_n = U_n/n$. We have $F_{V_n}(x) = \mathbb{P}(U_n/n \leq x) = \mathbb{P}(U_n \leq nx)$. Hence

- If $x \leq 0$, then $F_{V_n}(x) = 0 = F_U(x)$, since $U_n \geq 1$.
- If $x > 1$, then $F_{V_n}(x) = 1 = F_U(x)$, since $U_n \leq n$.
- If $0 < x \leq 1$, then $F_{V_n}(x) = \frac{\lfloor nx \rfloor}{n}$. Since $x - \frac{1}{n} \leq \frac{\lfloor nx \rfloor}{n} \leq x$, we have $F_{V_n}(x) \rightarrow x = F_U(x)$.

□

Proof of Lemma 4.13. We first show that

$$\left(\frac{U_n}{n+2}\right)^\alpha X_{U_n}^{(1)} + \left(\frac{n+1-U_n}{n+2}\right)^\alpha X_{n+1-U_n}^{(2)} \xrightarrow{d} U^\alpha X^{(1)} + (1-U)^\alpha X^{(2)}.$$

By Lemma A.3 and Slutsky's theorem,

$$\frac{U_n}{n+2} = \frac{U_n}{n} \cdot \frac{n}{n+2} \xrightarrow{d} U.$$

To apply the Continuous Mapping Theorem, we need to show that

$$\left(\frac{U_n}{n+2}, X_{U_n}^{(1)}, X_{n+1-U_n}^{(2)}\right) \xrightarrow{d} (U, X^{(1)}, X^{(2)}).$$

Since $U, X^{(1)}, X^{(2)}$ are independent, this is equivalent to showing that $\forall u, x_1, x_2 \in \mathbb{R}$

$$\mathbb{P}\left(U_n \leq \lfloor nu \rfloor, X_{U_n}^{(1)} \leq x_1, X_{n+1-U_n}^{(2)} \leq x_2\right) \longrightarrow \mathbb{P}(U \leq u) \mathbb{P}(X^{(1)} \leq x_1) \mathbb{P}(X^{(2)} \leq x_2).$$

Indeed,

$$\begin{aligned}
& \mathbb{P}\left(U_n \leq \lfloor nu \rfloor, X_{U_n}^{(1)} \leq x_1, X_{n+1-U_n}^{(2)} \leq x_2\right) \\
&= \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P}(U_n = k) \mathbb{P}\left(X_k^{(1)} \leq x_1, X_{n+1-k}^{(2)} \leq x_2 | U_n = k\right) \\
&= \frac{1}{n} \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P}\left(X_k^{(1)} \leq x_1, X_{n+1-k}^{(2)} \leq x_2\right) \quad (U_n \perp (X_k^{(1)}, X_{n+1-k}^{(2)})) \\
&= \frac{1}{n} \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P}\left(X_k^{(1)} \leq x_1\right) \mathbb{P}\left(X_{n+1-k}^{(2)} \leq x_2\right) \quad (X_k^{(1)} \perp X_{n+1-k}^{(2)})
\end{aligned}$$

Let F_n be the common CDF of $X_n^{(1)}$ and $X_n^{(2)}$, and F be the CDF of $X^{(1)}$. Let $m = \lfloor nu \rfloor$. The problem becomes showing that

$$\frac{1}{n} \sum_{k=1}^m F_k(x_1) F_{n+1-k}(x_2) \longrightarrow u F(x_1) F(x_2).$$

Equivalently,

$$\frac{m}{n} \left(\frac{1}{m} \sum_{k=1}^m (F_k(x_1) F_{n+1-k}(x_2) - F(x_1) F(x_2)) \right) \longrightarrow 0.$$

We have

$$\begin{aligned}
& \frac{1}{m} \sum_{k=1}^m |F_k(x_1) F_{n+1-k}(x_2) - F(x_1) F(x_2)| \\
& \leq \frac{1}{m} \sum_{k=1}^m |F_k(x_1) (F_{n+1-k}(x_2) - F(x_2))| + \frac{1}{m} \sum_{k=1}^m |F(x_2) (F_k(x_1) - F(x_1))| \\
& \leq \frac{1}{m} \sum_{k=1}^m |F_{n+1-k}(x_2) - F(x_2)| + \frac{1}{m} \sum_{k=1}^m |F_k(x_1) - F(x_1)| \quad (|F_n| \leq 1)
\end{aligned}$$

The second term goes to zero by the fact that $F_n(x_1) \rightarrow F(x_1)$ and Cesàro Means. We can write the first term as

$$\frac{1}{m} \sum_{k=n-m+1}^n |F_k(x_2) - F(x_2)|.$$

If $u = 1$, its convergence is similar to the second term. If $u < 1$, then $n - m + 1 \rightarrow \infty$. By pointwise convergence, $\forall \epsilon > 0, \exists N, \forall n \geq N : |F_n(x_2) - F(x_2)| < \epsilon$. Hence, for $n - m + 1 \geq N$,

$$\frac{1}{m} \sum_{k=n-m+1}^n |F_k(x_2) - F(x_2)| < \epsilon.$$

Using Slutsky's theorem again, we have

$$\left(\frac{n+1}{n+2}, \frac{U_n}{n+2}, X_{U_n}^{(1)}, X_{n+1-U_n}^{(2)} \right) \xrightarrow{d} \left(1, U, X^{(1)}, X^{(2)} \right).$$

A.2 Remaining proofs

Now define the function $f : \mathbb{R}_+^4 \cap \{(t, u, x, y) : t \geq u\} \rightarrow \mathbb{R}$ by

$$f(t, u, x, y) = u^\alpha x + (t - u)^\alpha y.$$

Then f is continuous. The Continuous Mapping Theorem applies. The convergence

$$\max \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)}, \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \xrightarrow{d} \max \left(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)} \right)$$

is similar. And finally, we define the continuous function $g(i, j, u, v) = iu + jv$ on $\mathbb{R}^4$ to finish the proof. $\square$

A.2.2 Proposition 4.14

Proof. The function f is strictly decreasing in α . Next, we show that f is continuous. The first term is continuous. We show that the second term is Lipschitz continuous. Let $\alpha, \alpha' \in (0, 1]$ and assume without loss of generality that $\alpha > \alpha'$. We have

$$\begin{aligned} & \mathbb{E} \left[\max \left(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)} \right) - \max \left(U^{\alpha'} X^{(1)}, (1-U)^{\alpha'} X^{(2)} \right) \right] \\ & \leq \mathbb{E} \left[\max \left(\left| U^\alpha - U^{\alpha'} \right| X^{(1)}, \left| (1-U)^\alpha - (1-U)^{\alpha'} \right| X^{(2)} \right) \right] \\ & \leq \mathbb{E} \left[\left| U^\alpha - U^{\alpha'} \right| \right] + \mathbb{E} \left[\left| (1-U)^\alpha - (1-U)^{\alpha'} \right| \right] \\ & = 2\mathbb{E} \left[\left| U^\alpha - U^{\alpha'} \right| \right] \quad (U \stackrel{d}{=} 1-U) \\ & = 2\mathbb{E} \left[U^{\alpha'} - U^\alpha \right] \quad (U^\alpha \geq U^{\alpha'}, a.s.) \\ & = \frac{2}{\alpha' + 1} - \frac{2}{\alpha + 1} = \frac{2(\alpha - \alpha')}{(\alpha + 1)(\alpha' + 1)} \\ & \leq 2(\alpha - \alpha'). \end{aligned}$$

Finally, we evaluate f at the endpoints.

$$\begin{aligned} \lim_{\alpha \rightarrow 0+} f(\mu, \alpha) &= 2u + v \mathbb{E} \left[\max \left(X^{(1)}, X^{(2)} \right) \right] > 2u + v > 1, \\ f(1) &= u + v \mathbb{E} \left[\max \left(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)} \right) \right] \leq u + v \leq 1. \end{aligned}$$

$\square$

A.2.3 Proposition 4.16

Proof. Let $\mu \in \mathcal{D}_2(1)$ have finite second moment, and let $Z \sim \mu$. Following the steps in the previous section, we have

$$\begin{aligned} \mathbb{E} [\Phi(Z)^2] &\leq \frac{2(u+v)}{2\alpha+1} \mathbb{E} [Z^2] + 2uB(\alpha+1, \alpha+1) \\ &< \frac{2(u+v)}{2(2u+v)-1} \mathbb{E} [Z^2] + 2uB(2u+v, 2u+v) \end{aligned}$$

Choose

$$M_0 = 2uB(2u+v, 2u+v) \frac{2(2u+v)-1}{2u-1}.$$

We have $\mathbb{E} [\Phi(Z)^2] \leq M$. Hence, Φ maps $\mathcal{D}_2(1, M)$ into itself. $\square$

A.2.4 Proposition 4.18

Proof. The set $\mathcal{D}_2(1, M)$ is nonempty since it contains δ_1 .

For convexity, let $\mu, \nu \in \mathcal{D}_2(1, M)$. Then for every $\lambda \in [0, 1]$, we have

$$\begin{aligned} \int_0^\infty x d(\lambda\mu + (1-\lambda)\nu) &= \lambda \int_0^\infty x d\mu + (1-\lambda) \int_0^\infty x d\nu = 1, \\ \int_0^\infty x^2 d(\lambda\mu + (1-\lambda)\nu) &= \lambda \int_0^\infty x^2 d\mu + (1-\lambda) \int_0^\infty x^2 d\nu \leq M. \end{aligned}$$

For compactness, we use Prokhorov's theorem to reduce to proving that $\mathcal{D}_2(1, M)$ is tight and closed. Tightness is guaranteed by Chebyshev's inequality: for any $\mu \in \mathcal{D}_2(1, M)$ and any $R > 0$

$$\mu(\{x \geq R\}) \leq \frac{1}{R^2} \int_0^\infty x^2 d\mu \leq \frac{M}{R^2}. \quad (1.2)$$

For closedness, let (μ_n) be a sequence in $\mathcal{D}_2(1, M)$ that converges weakly to μ . We show that $\mu \in \mathcal{D}_2(1, M)$. The boundedness of the second moment follows from a generalized Fatou's lemma [5, Theorem 1.1]

$$\int_0^\infty x^2 d\mu \leq \liminf_{n \rightarrow \infty} \int_0^\infty x^2 d\mu_n \leq M.$$

The function $x \mapsto x$ is uniformly integrable with respect to $\mathcal{D}_2(1, M)$ as follows. For any $\mu \in \mathcal{D}_2(1, M)$ and any $R > 0$

$$\int_R^\infty x d\mu \leq \int_R^\infty x \left(\frac{x}{R}\right) d\mu = \frac{1}{R} \int_R^\infty x^2 d\mu \leq \frac{1}{R} \int_0^\infty x^2 d\mu \leq \frac{M}{R}.$$

which tends to 0 as $R \rightarrow \infty$. Therefore, the first moment converges [2, Theorem 3.5]

$$\int_0^\infty x d\mu_n \rightarrow \int_0^\infty x d\mu.$$

□

A.2.5 Proposition 4.19

Lemma A.4 *For every $\mu, \nu \in \mathcal{D}_2(1)$ and $\alpha \in (0, 1]$, we have*

$$d_2(T_\alpha(\mu), T_\alpha(\nu)) \leq \sqrt{\frac{2(u+v)}{2\alpha+1}} d_2(\mu, \nu) \text{ and } d_1(T_\alpha(\mu), T_\alpha(\nu)) \leq \frac{2(u+v)}{\alpha+1} d_1(\mu, \nu).$$

Proof. Since I and J are Bernoulli, we have $I^2 = I$ and $J^2 = J$. Also, from $I + J \leq 1$, we have $IJ = 0$. Therefore, for every $X, X^{(1)}, X^{(2)} \stackrel{\text{iid}}{\sim} \mu$ and $Y, Y^{(1)}, Y^{(2)} \stackrel{\text{iid}}{\sim} \nu$, we have

$$\begin{aligned} d_2^2(T_\alpha(\mu), T_\alpha(\nu)) &\leq \mathbb{E} \left[I \left(\left(U^\alpha (X^{(1)} - Y^{(1)}) + (1-U)^\alpha (X^{(2)} - Y^{(2)}) \right)^2 \right) \right] \\ &\quad + \mathbb{E} \left[J \left(\max \left(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)} \right) - \max \left(U^\alpha Y^{(1)}, (1-U)^\alpha Y^{(2)} \right) \right)^2 \right] \\ &:= A + B. \end{aligned}$$

A.2 Remaining proofs

For the first expectation A , we use $\mathbb{E}[X] = \mathbb{E}[Y] = 1$ and get

$$\begin{aligned} A &= u\mathbb{E} \left[U^{2\alpha} \left(X^{(1)} - Y^{(1)} \right)^2 + (1-U)^{2\alpha} \left(X^{(2)} - Y^{(2)} \right)^2 \right] \\ &= \frac{2u}{2\alpha+1} \mathbb{E}[(X-Y)^2]. \end{aligned}$$

For the second one, B , we use the inequality

$$(\max(a, b) - \max(c, d))^2 \leq \max(a-c, b-d)^2 \leq (a-c)^2 + (b-d)^2$$

for real numbers a, b, c, d and get

$$\begin{aligned} B &\leq v\mathbb{E} \left[U^{2\alpha} \left(X^{(1)} - Y^{(1)} \right)^2 + (1-U)^{2\alpha} \left(X^{(2)} - Y^{(2)} \right)^2 \right] \\ &= \frac{2v}{2\alpha+1} \mathbb{E}[(X-Y)^2]. \end{aligned}$$

Therefore,

$$d_2^2(T_\alpha(\mu), T_\alpha(\nu)) \leq \frac{2(u+v)}{2\alpha+1} \mathbb{E}[(X-Y)^2].$$

Taking the expectation over $X \sim \mu$ and $Y \sim \nu$, we get the desired result. The case for d_1 is similar. $\square$

Proof of Proposition 4.19. Let (μ_n) be a sequence in $\mathcal{D}_2(1, M)$ that converges weakly to μ , which is equivalent to $d_1(\mu_n, \mu) \rightarrow 0$. To show that $\Phi(\mu_n) \xrightarrow{w} \Phi(\mu)$, we will show that $d_1(\Phi(\mu_n), \Phi(\mu)) \rightarrow 0$ by showing that Φ is Lipschitz with respect to d_1 . Let $\mu, \nu \in \mathcal{D}_2(1, M)$. Write $\alpha_1 := \alpha(\mu)$ and $\alpha_2 := \alpha(\nu)$. Without loss of generality, assume that $\alpha_1 > \alpha_2$. We have

$$\begin{aligned} d_1(\Phi(\mu), \Phi(\nu)) &= d_1(T_{\alpha_1}(\mu), T_{\alpha_2}(\nu)) \\ &\leq d_1(T_{\alpha_2}(\mu), T_{\alpha_2}(\nu)) + d_1(T_{\alpha_1}(\mu), T_{\alpha_2}(\mu)). \end{aligned}$$

By Lemma A.4, we have

$$d_1(T_{\alpha_2}(\mu), T_{\alpha_2}(\nu)) \leq \frac{2(u+v)}{\alpha_2+1} d_1(\mu, \nu) \leq 2(u+v)d_1(\mu, \nu).$$

For the second distance, we let $Y^{(1)}, Y^{(2)} \sim \nu$. Then, following the same steps as in Lemma 4.14, we have

$$d_1(T_{\alpha_1}(\mu), T_{\alpha_2}(\mu)) \leq 2(u+v)(\alpha_1 - \alpha_2).$$

We have $f(\mu, \alpha_1) = f(\nu, \alpha_2) = 1$. Hence

$$|f(\mu, \alpha_1) - f(\mu, \alpha_2)| = |f(\mu, \alpha_2) - f(\nu, \alpha_2)|.$$

Consider the left-hand side term where we fix the distribution μ . We rewrite f as

$$f(\mu, \alpha) = \frac{2u}{\alpha+1} + v\mathbb{E} \left[U^\alpha Z^{(1)} \mathbf{1}_A + (1-U)^\alpha Z^{(2)} \mathbf{1}_{A^c} \right], \quad Z^{(1)}, Z^{(2)} \stackrel{\text{iid}}{\sim} \mu,$$

where A is the event that $U^\alpha Z^{(1)} \geq (1-U)^\alpha Z^{(2)}$. Since $\mathbb{P}(U^\alpha Z^{(1)} = (1-U)^\alpha Z^{(2)}) = 0$, we have

$$\frac{\partial}{\partial \alpha} f(\mu, \alpha) = -\frac{2u}{(\alpha+1)^2} + v\mathbb{E} \left[\ln(U) U^\alpha Z^{(1)} \mathbf{1}_A + \ln(1-U) (1-U)^\alpha Z^{(2)} \mathbf{1}_{A^c} \right].$$

Since $\ln(U) < 0$ and $\ln(1-U) < 0$, we have for every $\alpha \in (0, 1]$,

$$\left| \frac{\partial}{\partial \alpha} f(\mu, \alpha) \right| \geq \frac{2u}{(\alpha+1)^2}.$$

A.2 Remaining proofs

By Mean Value Theorem, we have

$$|f(\mu, \alpha_1) - f(\mu, \alpha_2)| \geq \frac{2u}{(\alpha_1 + 1)^2} |\alpha_1 - \alpha_2| \geq \frac{u}{2} |\alpha_1 - \alpha_2|$$

We also have

$$|f(\mu, \alpha_2) - f(\nu, \alpha_2)| \leq \frac{2v}{\alpha_2 + 1} d_1(\mu, \nu) \leq 2v d_1(\mu, \nu).$$

Thus,

$$d_1(\Phi(\mu), \Phi(\nu)) \leq 2(u + v) \left(1 + \frac{4v}{u}\right) d_1(\mu, \nu).$$

□



Bibliography

- [1] Ricardo Baeza-Yates and Gaston H Gonnet. “A new approach to text searching”. In: *Communications of the ACM* 35.10 (1992), pp. 74–82.
- [2] Patrick Billingsley. *Convergence of Probability Measures*. Vol. 100. Wiley Online Library, 1999, pp. 80–120.
- [3] R Creighton Buck. “Bounded continuous functions on a locally compact space.” In: *Michigan Mathematical Journal* 5.2 (1958), pp. 95–104.
- [4] Cisco Systems. *Snort Rules Database*. <https://www.snort.org/>. Accessed: 2026-05-18. 2026.
- [5] Eugene A Feinberg, Pavlo O Kasyanov, and Nina V Zadoianchuk. “Fatou’s lemma for weakly converging probabilities”. In: *Theory of Probability & Its Applications* 58.4 (2014), pp. 683–689.
- [6] Philippe Flajolet and Robert Sedgewick. *Analytic combinatorics*. Vol. 1. cambridge University press Cambridge, 2009.
- [7] Victor Mikhaylovich Glushkov. “The abstract theory of automata”. In: *Russian Mathematical Surveys* 16.5 (1961), pp. 1–53.
- [8] Jan Holub, William F Smyth, and Shu Wang. “Fast pattern-matching on indeterminate strings”. In: *Journal of Discrete Algorithms* 6.1 (2008), pp. 37–50.
- [9] Donald E Knuth. *The Art of Computer Programming: Fundamental Algorithms, Volume 1*. Addison-Wesley Professional, 1997.
- [10] Florent Koechlin, Cyril Nicaud, and Pablo Rotondo. “Uniform random expressions lack expressivity”. In: *44th International Symposium on Mathematical Foundations of Computer Science (MFCS 2019)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik. 2019, pp. 51–1.
- [11] Florent Koechlin and Pablo Rotondo. “Analysis of an efficient reduction algorithm for random regular expressions based on universality detection”. In: *International Computer Science Symposium in Russia*. Springer. 2021, pp. 206–222.
- [12] Ralph Neininger and Ludger Rüschendorf. “A general limit theorem for recursive algorithms and combinatorial structures”. In: *The Annals of Applied Probability* 14.1 (2004), pp. 378–418.
- [13] Kun Qiu et al. “Teddy: An efficient simd-based literal matching engine for scalable deep packet inspection”. In: *Proceedings of the 50th International Conference on Parallel Processing*. 2021, pp. 1–11.
- [14] Michael Sipser. *Introduction to the Theory of Computation*. 3rd. Cengage Learning, 2012. ISBN: 978-1133187790.

-
- [15] Veeravalli S Varadarajan. “Weak convergence of measures on separable metric spaces”. In: *Sankhyā: The Indian Journal of Statistics (1933-1960)* 19.1/2 (1958), pp. 15–22.
 - [16] Vectorcamp. *Vectorscan: A portable fork of the high-performance regular expression matching library*. <https://github.com/Vectorcamp/vectorscan>. 2023.
 - [17] Xiang Wang et al. “Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}”. In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 2019, pp. 631–648.
 - [18] Hao Xu et al. “Harry: A scalable SIMD-based multi-literal pattern matching engine for deep packet inspection”. In: *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE. 2023, pp. 1–10.
 - [19] Eberhard Zeidler. *Nonlinear functional analysis and its applications: I: Fixed-point theorems*. Springer-Verlag New York Inc., 1986.
 - [20] VM Zolotarev. “Approximation of distributions of sums of independent random variables with values in infinite-dimensional spaces”. In: *Theory of Probability & Its Applications* 21.4 (1977), pp. 721–737.